

ส่วน แผนการสอน

- 1.วัตถุประสงค์
- 2.กลุ่มเป้าหมาย
- 3.ความรู้พื้นฐาน
- 4.เนื้อหา โดยย่อ กระชับ เน้น วัตถุประสงค์ ประโยชน์ของการใช้

ส่วน เอกสาร

- 1.สร้างบทนำ
- 2.สร้างบทนิยาม
- 3.สร้างบทหัวข้อ
- 5.ออกแบบคู่มือ
- 6.ออกแบบ workflow
- 7.TASK LIST Template
- 8.CHECKLIST Template
- 9.สรุป

- 1.บทนำ
- 2.บทนิยาม ศัพท์ ทุกส่วน

เช่น

Queue Processor คืออะไร

Queue Processor มีกี่แบบ

Queue Processor ใช้อย่างไร นำในกรณีไหน ทำไม่ต้องใช้ ประโยชน์ที่ได้รับ

3.ออกแบบ workflow

- วาดรูป dataflow สร้างรูปแบบ draw.io เหมือนจริง ลักษณะ flowchart TB เพื่ออธิบายกระบวนการทำความเข้าใจ
- พร้อมอธิบาย แบบละเอียด
- ยกตัวอย่างการใช้งานจริง หรือ กรณีศึกษา
- เหมเพลตและตัวอย่างโค้ด พร้อมนำไป run ได้ทันที มีคำอธิบายการใช้งานแต่ละจุด การคอมเม้น ภาษาไทย และ ภาษาอังกฤษ

4.ทำบทสรุปท้ายบท

-ประโยชน์ที่ได้รับ

-ข้อควรระวัง

-ข้อดี

-ข้อเสีย

-ข้อห้าม ถ้ามี

-แหล่ง อ้างอิง ที่มา

5.คู่มือ

-TASK LIST Template

-กระบวนการทำงานแต่ละขั้นตอน

-กระบวนการตรวจสอบ การทำงานแต่ละขั้นตอน

-กระบวนการ อธิบายปัญหา สรุป

- Root Cause Analysis (RCA) คือแนวทางหรือกระบวนการวิเคราะห์ปัญหาอย่างเป็นระบบ เพื่อค้นหา "สาเหตุที่แท้จริง" (Root Cause) ของปัญหาที่เกิดขึ้น

แทนที่จะแก้ไขเพียงอาการ (Symptoms) ที่ปรากฏ ช่วยป้องกันปัญหาไม่ให้เกิดซ้ำอย่างยั่งยืน

โดยนิยมใช้เครื่องมืออย่างเทคนิค 5 Why หรือแผนภูมิแก๊งปลาช่วยในการวิเคราะห์

แนวทางและขั้นตอนการวิเคราะห์ Root Cause

กำหนดปัญหาให้ชัดเจน (Define the Problem): ระบุสิ่งที่เกิดขึ้นจริง ผลกระทบ และขอบเขตของปัญหาอย่าง

ชัดเจนและวัดผลได้

รวบรวมข้อมูล (Gather Data): หาหลักฐาน บันทึกข้อมูล หรือสัมภาษณ์ผู้เกี่ยวข้อง เพื่อทำความเข้าใจสถานการณ์

วิเคราะห์สาเหตุ (Analyze the Root Cause): ใช้เทคนิค เช่น "5 Why" (ถาม "ทำไม" เพื่อเจาะลึก 5 ครั้ง) หรือ แผนภูมิกระดูกปลา (Fishbone Diagram)

เพื่อแยกแยะสาเหตุที่เป็นไปได้จนถึงสาเหตุรากฐาน

หาวิธีแก้ไข (Implement Solutions): กำหนดแนวทางแก้ไขที่ตรงจุดเพื่อกำจัดต้นตอของปัญหา

ติดตามผล (Monitor): ติดตามและประเมินผลลัพธ์เพื่อยืนยันว่าการแก้ไขได้ผลและปัญหาไม่กลับมาเกิดขึ้นซ้ำ

- ประโยชน์ของการใช้ RCA

แก้ไขปัญหาได้อย่างยั่งยืน: ป้องกันไม่ให้เกิดปัญหาเดิมเกิดขึ้นซ้ำอีก

ลดต้นทุนและความสูญเสีย: ไม่ต้องเสียเวลาและค่าใช้จ่ายในการแก้ไขปัญหาเดิมๆ ซ้ำซาก

ปรับปรุงกระบวนการ: ช่วยให้เห็นจุดอ่อนในกระบวนการทำงานและแก้ไขให้ดีขึ้น

-CHECKLIST Template เช่น

-กระบวนการทำงานแต่ละขั้นตอน

-กระบวนการตรวจสอบ การทำงานแต่ละขั้นตอน

Golang เล่ม 1

ครอบคลุมทุกมิติ ตั้งแต่พื้นฐานสู่สถาปัตยกรรมระดับองค์กร พร้อมแผนภาพและโค้ดตัวอย่างที่รันได้จริง

Version 3.0 – เมษายน 2026

สารบัญ

ภาคที่ 1: ประมวลกับการเขียนโปรแกรม

- บทที่ 1: ความรู้เบื้องต้นเกี่ยวกับการเขียนโปรแกรมคอมพิวเตอร์
- บทที่ 2: รู้จักกับภาษา Go
- บทที่ 3: พื้นฐานการใช้งาน Terminal
- บทที่ 4: เตรียมสภาพแวดล้อมสำหรับพัฒนา
- บทที่ 5: สร้างแอปพลิเคชันแรกของคุณ

ภาคที่ 2: พื้นฐานภาษาและโครงสร้างข้อมูล

- บทที่ 6: ระบบเลขฐานสองและฐานสิบ
- บทที่ 7: เลขฐานสิบหก, ฐานแปด, ASCII, UTF8, Unicode และ Runes
- บทที่ 8: ตัวแปร, ค่าคงที่ และชนิดข้อมูลพื้นฐาน
- บทที่ 9: คำสั่งควบคุมการทำงาน
- บทที่ 10: ฟังก์ชัน
- บทที่ 11: แพคเกจและการนำเข้า
- บทที่ 12: การเริ่มต้นทำงานของแพคเกจ
- บทที่ 13: การสร้างชนิดข้อมูลใหม่ (Types)
- บทที่ 14: เมธอด (Methods)
- บทที่ 15: พอยน์เตอร์ (Pointer)
- บทที่ 16: อินเทอร์เฟซ (Interfaces)

ภาคที่ 3: การจัดการโปรเจกต์และโครงสร้างข้อมูลขั้นสูง

- บทที่ 17: Go Modules - การจัดการโปรเจกต์สมัยใหม่
- บทที่ 18: Go Module Proxies
- บทที่ 19: การทดสอบหน่วย (Unit Tests)
- บทที่ 20: อาร์เรย์ (Arrays)
- บทที่ 21: สไลซ์ (Slices)
- บทที่ 22: แมพ (Maps)
- บทที่ 23: การจัดการข้อผิดพลาด (Errors)

ภาคที่ 4: การพัฒนาแอปพลิเคชันเชิงปฏิบัติ

- บทที่ 24: ฟังก์ชันนิรนาม (Anonymous functions) และ Closure
- บทที่ 25: การจัดการข้อมูล JSON และ XML
- บทที่ 26: พื้นฐานการสร้าง HTTP Server
- บทที่ 27: Enum, iota และ Bitmask
- บทที่ 28: วันที่และเวลา
- บทที่ 29: การจัดเก็บข้อมูล: ไฟล์และฐานข้อมูล
- บทที่ 30: การทำงานพร้อมกัน (Concurrency)
- บทที่ 31: การบันทึกเหตุการณ์ (Logging)
- บทที่ 32: เทมเพลต (Templates)
- บทที่ 33: การจัดการค่า Configuration

ภาคที่ 5: สู่การเป็นนักพัฒนา Go มืออาชีพ

- บทที่ 34: การวัดประสิทธิภาพ (Benchmarks)
- บทที่ 35: สร้าง HTTP Client
- บทที่ 36: การวิเคราะห์โปรไฟล์ (Program Profiling)
- บทที่ 37: การจัดการ Context
- บทที่ 38: Generics - การเขียนโค้ดแบบยืดหยุ่น
- บทที่ 39: Go กับกระบวนทัศน์ OOP?
- บทที่ 40: การอัปเดตหรือดาวน์เกรดเวอร์ชัน Go
- บทที่ 41: คำแนะนำในการออกแบบโค้ดที่ดี
- บทที่ 42: ชีทสรุป (Cheatsheet)

ภาคที่ 6: เครื่องมือและไลบรารียอดนิยม

- บทที่ 43: chi, viper, cobra, zap และเครื่องมือสำคัญ
- บทที่ 44: GORM – ORM ทรงพลังสำหรับ Go
- บทที่ 45: การส่งอีเมลด้วย gomail และ hermes

ภาคที่ 7: การออกแบบสถาปัตยกรรมและ Workflow

- บทที่ 46: Clean Architecture และโครงสร้างโปรเจกต์
- บทที่ 47: Blueprint สำหรับโปรเจกต์ Go ระดับ Production
- บทที่ 48: การออกแบบ Workflow และ Task Management

ภาคที่ 8: Domain-Driven Design (DDD) กับ Go

- บทที่ 49: หลักการ DDD และการนำไปใช้ใน Go
- บทที่ 50: Aggregates, Event Storming และ CQRS
- บทที่ 51: การออกแบบบริการด้วย Go-DDD

ภาคที่ 9: การผสานระบบภายนอกและคุณลักษณะเสริม (Advanced Integrations)

- บทที่ 52: Redis สำหรับ Cache และ Message Queue
- บทที่ 53: RabbitMQ – Message Broker มาตรฐานองค์กร
- บทที่ 54: MQTT สำหรับ IoT และระบบเรียลไทม์
- บทที่ 55: InfluxDB – Time-Series Database
- บทที่ 56: WebSocket และ Socket.IO
- บทที่ 57: การส่ง SMS และ LINE Notify
- บทที่ 58: Discord Webhook สำหรับแจ้งเตือน

ภาคที่ 10: เทมเพลต กระบวนการพัฒนา และตัวอย่างโค้ด

- บทที่ 59: ตัวอย่างโค้ดครบวงจร (Full-stack Example)
- บทที่ 60: Task List Template
- บทที่ 61: Checklist Template
- บทที่ 62: แผนภาพการทำงาน (Workflow Diagram)
- บทที่ 63: mop Config – การจัดการ Configuration

ภาคผนวก

- GORM CRUD กับฐานข้อมูลหลายประเภท
- คู่มือภาษา Go ฉบับ นำไปทำงาน

ภาคที่ 1: ปฐมบทกับการเขียนโปรแกรม

บทที่ 1: ความรู้เบื้องต้นเกี่ยวกับการเขียนโปรแกรมคอมพิวเตอร์

1.1 การเขียนโปรแกรมคืออะไร?

การเขียนโปรแกรม (Programming) คือกระบวนการสร้างชุดคำสั่งที่ใช้ควบคุมการทำงานของคอมพิวเตอร์ให้ทำงานตามที่เราต้องการ โดยใช้ภาษาเฉพาะที่คอมพิวเตอร์สามารถเข้าใจได้ ภาษาที่มนุษย์ใช้เขียนเรียกว่า "ภาษาคอมพิวเตอร์ระดับสูง" (High-Level Language) เช่น Go, Python, Java ซึ่งจากนั้นจะถูกแปลงเป็นภาษาเครื่อง (Machine Language) ที่เป็นเลขฐานสอง (0 และ 1) ที่ชิปสามารถประมวลผลได้

1.2 โครงสร้างพื้นฐานของโปรแกรม

โปรแกรมคอมพิวเตอร์โดยทั่วไปประกอบด้วย:

- **ข้อมูล (Data)** : ตัวเลข, ข้อความ, รายการต่างๆ
- **การประมวลผล (Processing)** : การดำเนินการกับข้อมูล เช่น การคำนวณ การเปรียบเทียบ

- การควบคุมการทำงาน (**Control Flow**) : การตัดสินใจ (if-else), การวนซ้ำ (loop)
- การจัดเก็บ (**Storage**) : หน่วยความจำ, ไฟล์, ฐานข้อมูล
- อินพุต/เอาต์พุต (**I/O**) : การรับข้อมูลจากผู้ใช้ หรือแสดงผล

1.3 ตัวแปลภาษาและคอมไพเลอร์

- **คอมไพเลอร์ (Compiler)** : แปลงซอร์สโค้ดทั้งหมดเป็นไฟล์ไบนารีได้ก่อนรัน (เช่น Go, C)
- **อินเทอร์พรีเตอร์ (Interpreter)** : แปลงและรันทีละคำสั่ง (เช่น Python, JavaScript)

Go เป็นภาษาแบบคอมไพล์ (compiled) ซึ่งมีข้อดีคือทำงานเร็วและสร้างไฟล์ binary ที่รันได้ทันทีโดยไม่ต้องพึ่งพาสิ่งแวดล้อมอื่น (ยกเว้นระบบปฏิบัติการ)

1.4 กระบวนทัศน์การเขียนโปรแกรม

Go รองรับการเขียนโปรแกรมแบบ:

- **Procedural** : ใช้ฟังก์ชันและลำดับขั้นตอน
- **Concurrent** : ทำงานพร้อมกันด้วย goroutine
- **Functional** (บางส่วน) : ฟังก์ชันเป็น first-class citizen
- **ไม่ใช่ OOP แบบคลาสสิก** : ใช้ struct และ interface แทน inheritance

1.5 Golang Procedural คืออะไร

Procedural หรือการเขียนโปรแกรมแบบเชิงกระบวนการ เป็นรูปแบบที่เน้นการเขียนฟังก์ชัน (function) และเรียกใช้ตามลำดับขั้นตอน โกลัง (Go) รองรับการเขียนแบบนี้โดยใช้ฟังก์ชันเป็นหน่วยหลัก โค้ดจะถูกแบ่งออกเป็นฟังก์ชันย่อย ๆ ที่ทำงานเฉพาะอย่าง และเรียกใช้ตามลำดับที่กำหนด ถึงแม้ Go จะไม่ใช่ภาษา procedural ล้วน ๆ แต่ก็สามารถเขียนในสไตล์นี้ได้เป็นอย่างดี

1.6 Concurrent คืออะไร

Concurrency (การทำงานร่วมกัน) คือความสามารถของโปรแกรมในการจัดการงานหลาย ๆ งานพร้อมกัน โดยงานเหล่านี้อาจเริ่มทำงาน สลับการทำงาน หรือจบลงในเวลาที่แตกต่างกัน โดยไม่จำเป็นต้องทำงานพร้อมกันจริง ๆ (parallel) Go มีเครื่องมือสำหรับ concurrency โดยตรง ได้แก่ **goroutine** และ **channel** ทำให้การเขียนโปรแกรม concurrent ง่ายและมีประสิทธิภาพ

1.7 goroutine คืออะไร

goroutine คือเธรด (thread) ขนาดเบาที่ถูกจัดการโดย runtime ของ Go ใช้สำหรับทำงานแบบ concurrent การสร้าง goroutine ทำได้ง่ายโดยใช้คีย์เวิร์ด **go** หน้าฟังก์ชัน:

```
go myFunction()
```

Goroutine มีน้ำหนักเบา ใช้สแต็กเริ่มต้นเพียงไม่กี่กิโลไบต์ และสามารถทำงานหลายหมื่นตัวพร้อมกันได้โปรแกรมเดียว

1.8 Functional คืออะไร

Functional programming (การเขียนโปรแกรมเชิงฟังก์ชัน) เป็นกระบวนทัศน์ที่เน้นการใช้ฟังก์ชันบริสุทธิ์ (pure function) ไม่มีการเปลี่ยนแปลงสถานะภายนอก และใช้ฟังก์ชันเป็นพลเมืองชั้นหนึ่ง (first-class citizen) Go รองรับคุณสมบัติบางอย่างของ functional เช่น การส่งฟังก์ชันเป็นค่าพารามิเตอร์ การคืนค่าฟังก์ชัน แต่ไม่มีการรับประกันความบริสุทธิ์ของฟังก์ชันหรือโครงสร้างข้อมูลแบบ immutable โดยตรง

1.9 first-class citizen คืออะไร

First-class citizen (พลเมืองชั้นหนึ่ง) หมายถึงสิ่งที่สามารถทำได้เช่นเดียวกับค่าอื่น ๆ ในภาษา เช่น ถูกกำหนดให้กับตัวแปร ถูกส่งเป็นอาร์กิวเมนต์ให้ฟังก์ชัน ถูกคืนค่าจากฟังก์ชัน หรือถูกเก็บในโครงสร้างข้อมูล ใน Go **ฟังก์ชัน** ถือเป็น first-class citizen:

```
var fn func(int) int = func(x int) int { return x * x }
```

นอกจากนี้ ชนิดข้อมูลต่าง ๆ เช่น struct, interface ก็ถือเป็น first-class citizen เช่นกัน

1.10 OOP คืออะไร

OOP (Object-Oriented Programming) เป็นกระบวนทัศน์ที่เน้นการสร้างวัตถุ (object) ซึ่งรวมข้อมูลและพฤติกรรมเข้าด้วยกัน หลักการสำคัญคือ encapsulation, inheritance, และ polymorphism Go ไม่ใช่ภาษา OOP แบบดั้งเดิมเพราะไม่มี class แต่ใช้ **struct** เป็นตัวเก็บข้อมูล และใช้ **method** ที่ผูกกับ struct รวมถึง **interface** เพื่อสร้าง polymorphism การสืบทอด (inheritance) ทำได้โดยการ **composition** (การแทรก struct) แทน

1.11 struct คืออะไร

struct เป็นชนิดข้อมูลที่ใช้รวมฟิลด์ (field) หลาย ๆ ชนิดเข้าด้วยกัน คล้ายกับ class ในภาษาอื่น แต่ไม่มี method ในตัว เราสามารถกำหนด method ให้กับ struct ได้โดยประกาศฟังก์ชันที่มี **receiver** เป็น struct นั้น

```
type Person struct {
    Name string
    Age  int
}

func (p Person) Greet() string {
    return "Hello, " + p.Name
}
```

1.12 interface คืออะไร

interface คือชนิดข้อมูลที่กำหนดชุดของ method signature (ชื่อฟังก์ชัน, พารามิเตอร์, ผลลัพธ์) โดยไม่ระบุการนำไปใช้ ถ้าชนิดใด (เช่น struct) มี method ครบตามที่ interface กำหนด ชนิดนั้นจะถูกพิจารณาว่า **implement** interface นั้นโดยอัตโนมัติ (implicit implementation) ทำให้เกิด polymorphism แบบหนึ่งใน Go

```
type Greeter interface {
    Greet() string
}

// Person มี method Greet() string ดังนั้น Person จึงเป็น Greeter โดยอัตโนมัติ
```

1.13 inheritance คืออะไร

Inheritance (การสืบทอด) คือกลไกที่ class หนึ่งสามารถสืบทอดสมาชิก (ฟิลด์และ method) จากอีก class หนึ่ง เพื่อนำมาใช้หรือขยายความสามารถ Go **ไม่มี** inheritance แบบ class-based แต่นิยมใช้ **composition** โดยการฝัง struct (embedded struct) เพื่อให้ได้ผลลัพธ์คล้ายการสืบทอด แต่ยังคงความยืดหยุ่นและหลีกเลี่ยงปัญหาที่เกิดจาก inheritance ที่ซับซ้อน

```
type Animal struct {
    Name string
}

type Dog struct {
    Animal // ฝัง struct Animal เข้ามา
    Breed string
}
```

Dog จะสามารถเข้าถึงฟิลด์ Name และ method (ถ้ามี) ของ Animal ได้โดยตรง

1. Procedural Programming

Procedural คืออะไร?

Procedural programming (การเขียนโปรแกรมแบบกระบวนการ) เป็นกระบวนทัศน์ที่เน้นการเขียนโค้ดเป็นชุดของคำสั่งหรือฟังก์ชัน (procedures / routines) ที่ทำงานตามลำดับขั้นตอน (step-by-step) โดยใช้ตัวแปร โครงสร้างควบคุม (if, for, switch) และฟังก์ชันที่เรียกซ้ำกันได้

Procedural มีกี่แบบ?

ในทางปฏิบัติ Procedural programming ไม่ได้แบ่งเป็น “แบบ” อย่างเป็นทางการ แต่สามารถมองได้ตาม **โครงสร้างของโค้ด**:

- **Linear / Sequential** – คำสั่งทำงานเรียงตามลำดับ
- **Modular** – แบ่งโค้ดออกเป็นฟังก์ชัน/โมดูล
- **Structured** – ใช้โครงสร้างควบคุม (sequence, selection, iteration) หลีกเลี่ยง goto

ภาษา Go มีพื้นฐานเป็น procedural โดยมีการจัดระเบียบผ่านฟังก์ชันและแพ็คเกจ

ใช้อย่างไร ในกรณีไหน?

- เขียนโปรแกรมที่มีขั้นตอนชัดเจน (linear flow)
- งานที่ต้องการประสิทธิภาพสูง ควบคุมทรัพยากรใกล้เคียงฮาร์ดแวร์
- โค้ดขนาดเล็กถึงกลาง ไม่ต้องการความซับซ้อนของ OOP
- Go มักใช้ procedural ร่วมกับ concurrent และ OOP (ผ่าน struct & interface)

หลักการทำงาน

1. โปรแกรมเริ่มทำงานจากฟังก์ชัน `main()`
2. เรียกฟังก์ชันตามลำดับ อาจมีการส่งค่าและรับค่ากลับ
3. ตัวแปรมีขอบเขตตาม block หรือ package
4. การทำงานเป็นแบบ imperative: เปลี่ยนแปลงสถานะของตัวแปรโดยตรง

Dataflow Diagram (Flowchart TB)

```
graph TD
    Start([เริ่ม main]) --> FuncA[เรียก Function A]
    FuncA --> FuncB[เรียก Function B]
    FuncB --> Cond{เงื่อนไข?}
    Cond -- true --> Action1[ทำ Action 1]
    Cond -- false --> Action2[ทำ Action 2]
    Action1 --> End1([จบ])
    Action2 --> End2([จบ])
```

ตัวอย่างการใช้งานจริง: โปรแกรมคำนวณราคาสินค้า

```
package main

import "fmt"

// ฟังก์ชันคำนวณราคารวม
func calculateTotal(price float64, quantity int) float64 {
    return price * float64(quantity)
}

// ฟังก์ชันคำนวณส่วนลด
func applyDiscount(total float64, discountPercent float64) float64 {
    return total * (1 - discountPercent/100)
}

// ฟังก์ชันหลัก
func main() {
    price := 250.0
    qty := 3
    discount := 10.0

    total := calculateTotal(price, qty)
    fmt.Printf("Total: %.2f\n", total)
```

```
finalPrice := applyDiscount(total, discount)
fmt.Printf("After discount: %.2f\n", finalPrice)
}
```

เทมเพลตโครงสร้างโปรเจกต์แบบ Procedural ใน Go

```
project/
├─ main.go
├─ handlers/      # ฟังก์ชันจัดการ request
├─ services/      # ฟังก์ชัน business logic
├─ repositories/  # ฟังก์ชันติดต่อฐานข้อมูล
└─ utils/         # ฟังก์ชันช่วยเหลือ
```

โค้ดในแต่ละ package จะเป็นชุดฟังก์ชันที่ทำงานตามลำดับที่เรียกใช้

2. Concurrent Programming

Concurrent คืออะไร?

Concurrent programming คือการเขียนโปรแกรมให้สามารถทำงานหลายอย่าง **พร้อมกัน** (overlap in time) โดยไม่จำเป็นต้องทำพร้อมกันจริง (parallel) แต่เป็นการจัดการหลาย tasks สลับกันเพื่อเพิ่มประสิทธิภาพและความสามารถในการตอบสนอง

Go มี **goroutine** (เธรดน้ำหนักเบา) และ **channel** สำหรับการสื่อสารระหว่าง goroutines ทำให้การเขียน concurrent โปรแกรมง่ายและปลอดภัย

Concurrent มีกี่แบบ?

รูปแบบหลักในการเขียน concurrent code:

- **Goroutines + Channels** – CSP (Communicating Sequential Processes) แบบ Go
- **Mutex / Atomic** – ใช้การล็อกเพื่อป้องกัน race condition
- **Worker Pool** – ใช้ goroutines จำนวนคงที่ทำงานจากคิวงาน
- **Pipeline** – ข้อมูลไหลผ่านหลาย goroutines ที่เชื่อมต่อกันด้วย channel

ใช้อย่างไร ในกรณีไหน?

- งานที่ต้องการทำหลายอย่างพร้อมกัน (web server, API calls)
- งาน I/O-bound (อ่านไฟล์, เรียกฐานข้อมูล, HTTP request)
- ปรับปรุงประสิทธิภาพการทำงานของระบบ
- Queue processor, background jobs

หลักการทำงาน

1. สร้าง goroutine ด้วย `go function()`

2. Goroutine ทำงาน concurrent กับ main goroutine
3. ใช้ `channel` สำหรับส่งข้อมูลระหว่าง goroutines (synchronization)
4. ใช้ `sync.WaitGroup` หรือ `context` เพื่อควบคุม lifecycle

Dataflow Diagram (Flowchart TB) - Worker Pool

```
graph TB
    subgraph Main
        A[main] --> B[สร้าง Jobs Channel]
        B --> C[สร้าง Workers n ตัว]
        C --> D[ส่งงานเข้า Jobs Channel]
    end

    subgraph Worker1
        E[for job := range jobs] --> F[ประมวลผล job]
    end

    subgraph Worker2
        G[for job := range jobs] --> H[ประมวลผล job]
    end

    D --> E
    D --> G
    F --> I[ส่งผลลัพธ์]
    H --> I
    I --> J[main รวบรวมผล]
```

ตัวอย่างการใช้งานจริง: Download เว็บหลาย ๆ ที่พร้อมกัน

```
package main

import (
    "fmt"
    "net/http"
    "sync"
    "time"
)

func fetch(url string, wg *sync.WaitGroup, results chan<- string) {
    defer wg.Done()
    start := time.Now()
    resp, err := http.Get(url)
    if err != nil {
        results <- fmt.Sprintf("%s -> error: %v", url, err)
        return
    }
    defer resp.Body.Close()
    elapsed := time.Since(start)
    results <- fmt.Sprintf("%s -> %d ms", url, elapsed.Milliseconds())
}
```

```

}

func main() {
    urls := []string{
        "https://google.com",
        "https://github.com",
        "https://stackoverflow.com",
    }

    var wg sync.WaitGroup
    results := make(chan string, len(urls))

    for _, url := range urls {
        wg.Add(1)
        go fetch(url, &wg, results)
    }

    wg.Wait()
    close(results)

    for res := range results {
        fmt.Println(res)
    }
}

```

เทมเพลต Worker Pool

```

type Job struct { ID int; Data interface{} }
type Result struct { JobID int; Output interface{}; Err error }

func worker(jobs <-chan Job, results chan<- Result, wg *sync.WaitGroup) {
    defer wg.Done()
    for job := range jobs {
        // process job
        results <- Result{JobID: job.ID, Output: processedData}
    }
}

func main() {
    jobs := make(chan Job, 100)
    results := make(chan Result, 100)

    // start workers
    var wg sync.WaitGroup
    for w := 0; w < 5; w++ {
        wg.Add(1)
        go worker(jobs, results, &wg)
    }

    // send jobs
}

```

```
for i := 0; i < 100; i++ {  
    jobs <- Job{ID: i, Data: ...}  
}  
close(jobs)  
wg.Wait()  
close(results)  
  
// collect results  
for res := range results { ... }  
}
```

3. Functional Programming

Functional คืออะไร?

Functional programming (การเขียนโปรแกรมเชิงฟังก์ชัน) เป็นกระบวนทัศน์ที่มองการคำนวณเป็นการประเมินค่าของฟังก์ชัน โดยเน้น:

- **Pure functions** – ผลลัพธ์ขึ้นอยู่กับอินพุตเท่านั้น ไม่มี side effect
- **Immutability** – ข้อมูลไม่เปลี่ยนแปลงหลังสร้าง
- **First-class functions** – ฟังก์ชันเป็นค่าที่สามารถส่งต่อได้
- **Higher-order functions** – ฟังก์ชันรับฟังก์ชันเป็นพารามิเตอร์หรือคืนฟังก์ชัน

Go รองรับฟังก์ชัน first-class, closures, และ higher-order functions แต่ไม่มี immutability บังคับ (ใช้แนวทาง pragmatic functional)

Functional มีกี่แบบ?

- **Pure FP** – ใช้เฉพาะ pure functions, immutable data (Haskell, Elm)
- **Impure FP** – อนุญาต side effect บางส่วน (Go, JavaScript, Scala)
- **Higher-order functions** – map, filter, reduce
- **Function composition** – นำฟังก์ชันเล็ก ๆ มาต่อกัน

ใช้อย่างไร ในกรณีไหน?

- การแปลงข้อมูล (transform) ด้วย pipeline
- เขียนโค้ดที่คาดเดาได้ง่าย ทดสอบง่าย
- ลดความซับซ้อนในการจัดการสถานะ
- ใช้ร่วมกับ concurrent (pure functions ปลอดภัยต่อ race condition)

หลักการทำงาน

1. กำหนดฟังก์ชันเล็ก ๆ ที่ทำหน้าที่เฉพาะ
2. ใช้ higher-order functions (map, filter, reduce) กับ slice
3. หลีกเลี่ยงการเปลี่ยนแปลงตัวแปรภายนอก (immutability เท่าที่ทำได้)
4. สร้าง pipeline โดยการต่อฟังก์ชันเข้าด้วยกัน

Dataflow Diagram (Flowchart TB) - Pipeline

```
graph LR
  A[Input Data] --> B[Filter]
  B --> C[Map]
  C --> D[Reduce]
  D --> E[Output Result]
```

ตัวอย่างการใช้งานจริง: ประมวลผลข้อมูลผู้ใช้

```
package main

import (
    "fmt"
    "strings"
)

// Pure function
func toUpper(s string) string { return strings.ToUpper(s) }

// Higher-order filter
func filter(users []string, predicate func(string) bool) []string {
    var result []string
    for _, u := range users {
        if predicate(u) {
            result = append(result, u)
        }
    }
    return result
}

// Higher-order map
func mapFunc(users []string, fn func(string) string) []string {
    result := make([]string, len(users))
    for i, u := range users {
        result[i] = fn(u)
    }
    return result
}

func main() {
    users := []string{"alice", "bob", "charlie", "admin"}

    // Pipeline: filter user with length > 3, then convert to upper
    filtered := filter(users, func(u string) bool { return len(u) > 3 })
    upper := mapFunc(filtered, toUpper)

    fmt.Println(upper) // [CHARLIE ADMIN]
}
```

เทมเพลต: generic map/filter/reduce (Go 1.18+)

```
type Slice[T any] []T

func (s Slice[T]) Map(fn func(T) T) Slice[T] {
    result := make(Slice[T], len(s))
    for i, v := range s {
        result[i] = fn(v)
    }
    return result
}

func (s Slice[T]) Filter(pred func(T) bool) Slice[T] {
    result := Slice[T]{}
    for _, v := range s {
        if pred(v) {
            result = append(result, v)
        }
    }
    return result
}
```

4. Object-Oriented Programming (OOP)

OOP คืออะไร?

Object-Oriented Programming (การเขียนโปรแกรมเชิงวัตถุ) เป็นกระบวนทัศน์ที่จัดระเบียบโค้ดเป็น "วัตถุ" (objects) ซึ่งรวมข้อมูล (fields) และพฤติกรรม (methods) เข้าด้วยกัน เน้นหลักการ:

- **Encapsulation** – ซ่อนรายละเอียดภายใน
- **Inheritance** – สืบทอดคุณสมบัติจากคลาสแม่ (Go ใช้ composition แทน)
- **Polymorphism** – หลายรูปแบบผ่าน interface

ภาษา Go ไม่มีคลาสแบบดั้งเดิม แต่ใช้ **struct** แทนข้อมูล และ **interface** แทนพฤติกรรมร่วมกัน ทำให้ได้แนวคิด OOP แบบ lightweight

OOP มีกี่แบบ?

ตามกระบวนทัศน์ OOP แบ่งเป็น:

- **Class-based** (Java, C++, Python) – มีคลาสเป็นต้นแบบ
- **Prototype-based** (JavaScript) – วัตถุสืบทอดจากวัตถุอื่น
- Go ใช้ **struct + interface** เข้าถึง OOP ในรูปแบบ composition over inheritance

ใช้อย่างไร ในกรณีไหน?

- โปรแกรมที่ต้องการจำลองสิ่งของในโลกจริง (entity modeling)
- ต้องการความสามารถในการขยาย (extension) ผ่าน interface
- ต้องการ polymorphism โดยไม่ต้องใช้ inheritance ซับซ้อน
- โค้ดขนาดใหญ่ที่ต้องการ modularity

หลักการทำงาน

1. กำหนด **struct** สำหรับเก็บข้อมูล
2. เพิ่ม **method** ให้ struct ด้วย `func (r Receiver) MethodName()`
3. กำหนด **interface** เพื่อประกาศชุดของ method ที่ต้องมี
4. ใช้งาน polymorphism ผ่าน interface (dependency injection, mock)

Dataflow Diagram (Flowchart TB) - Polymorphism

```
graph TB
    A[Interface Speaker] --> B1[Dog.Speak]
    A --> B2[Cat.Speak]
    A --> B3[Robot.Speak]
    C[Function makeSound] --> A
    C --> D[เรียก s.Speak]
```

ตัวอย่างการใช้งานจริง: สัตว์เลี้ยงและหุ่นยนต์

```
package main

import "fmt"

// Interface
type Speaker interface {
    Speak() string
}

// Struct Dog
type Dog struct{ Name string }

func (d Dog) Speak() string {
    return fmt.Sprintf("%s says Woof!", d.Name)
}

// Struct Cat
type Cat struct{ Name string }

func (c Cat) Speak() string {
    return fmt.Sprintf("%s says Meow!", c.Name)
}

// Function ที่ใช้ polymorphism
func makeSound(s Speaker) {
```

```

    fmt.Println(s.Speak())
}

func main() {
    dog := Dog{Name: "Rex"}
    cat := Cat{Name: "Luna"}

    makeSound(dog) // Rex says Woof!
    makeSound(cat) // Luna says Meow!
}

```

เทมเพลต: โครงสร้าง OOP ใน Go

```

// entity.go
type Entity struct {
    ID    int
    Name  string
}

func (e Entity) GetID() int { return e.ID }

// repository.go
type Repository interface {
    Save(entity Entity) error
    Find(id int) (Entity, error)
}

type MySQLRepository struct {
    db *sql.DB
}

func (r MySQLRepository) Save(entity Entity) error { ... }
func (r MySQLRepository) Find(id int) (Entity, error) { ... }

```

ขยายความ เพื่อให้เห็นภาพชัด

- Encapsulation คืออะไร
- Encapsulation มีกี่แบบ
- Encapsulation ใช้อย่างไร ในกรณีไหน ใช้อย่างไร แสดงรูปการทำงาน

หลักการ OOP ในภาษา Go (Golang)

หมายเหตุสำคัญ: Go ไม่ใช่ภาษา OOP เต็มรูปแบบ (ไม่มี class, ไม่มี inheritance แบบดั้งเดิม) แต่มีแนวคิดที่คล้ายคลึงกันผ่าน **struct**, **interface**, และ **composition (การซ้อน struct)** โดย Go สนับสนุน Encapsulation และ Polymorphism ได้ดี ส่วน Inheritance ใช้ **Embedding** แทน

📦 1. Encapsulation (การห่อหุ้มข้อมูล) ใน Go

🔍 Encapsulation คืออะไรใน Go?

Encapsulation ใน Go ทำได้โดยควบคุม **การมองเห็น (visibility)** ผ่าน **ตัวพิมพ์ใหญ่/เล็ก** ของชื่อฟิลด์หรือเมธอด:

- **ตัวพิมพ์ใหญ่ (Exported)** → เข้าถึงได้จากแพ็คเกจอื่น (public)
- **ตัวพิมพ์เล็ก (Unexported)** → เข้าถึงได้เฉพาะภายในแพ็คเกจเดียวกัน (private)

Go ไม่มี getter/setter อัตโนมัติ แต่เราสร้างฟังก์ชันหรือเมธอดเพื่อควบคุมการเข้าถึง

📊 Encapsulation มีกี่แบบใน Go?

แบ่งตามระดับการมองเห็นได้ **2 แบบ** (เรียบง่ายกว่า Java):

ระดับ	การตั้งชื่อ	การเข้าถึง
Exported (public)	ขึ้นต้นด้วยตัวพิมพ์ใหญ่ เช่น <code>Balance</code> , <code>GetBalance()</code>	ทุกแพ็คเกจ
Unexported (private)	ขึ้นต้นด้วยตัวพิมพ์เล็ก เช่น <code>balance</code> , <code>setBalance()</code>	เฉพาะแพ็คเกจเดียวกัน

🌀 ใช้อย่างไร? ในกรณีไหน?

- **ใช้เมื่อ** ต้องการควบคุมการแก้ไขข้อมูล ป้องกันสถานะไม่ถูกต้อง
- **กรณีใช้งานจริง:**
 - ระบบบัญชีธนาคาร (ห้ามให้ยอดติดลบ)
 - ระบบผู้ใช้ (ห้ามตั้งรหัสผ่านสั้นเกินไป)
 - ข้อมูลการตั้งค่า (config) ที่อ่านอย่างเดียว

📁 แผนภาพ Dataflow (Flowchart TB) – Encapsulation ใน Go

```
graph TD
    subgraph External ["แพ็คเกจอื่น (client)"]
        A[เรียกใช้เมธอด SetBalance / GetBalance]
    end

    subgraph Internal ["แพ็คเกจเดียวกัน (account)"]
        B["เมธอด Exported (พิมพ์ใหญ่)<br>func (a *Account) SetBalance(amt float64)"]
        C["ตรวจสอบความถูกต้อง<br>amt >= 0 ?"]
        D["ฟิลด์ unexported (พิมพ์เล็ก)<br>balance float64"]
        E["เมธอด GetBalance() float64"]
    end

    F[ส่งค่ากลับ / แสดงผล]

    A --> B
    B --> C
    C --> D
    D --> E
    E --> F
```

```
C -- ผ่าน --> D
C -- ไม่ผ่าน --> G[คืน error หรือ log]
D --> E
E --> F
G --> F
```

คำอธิบายแผนภาพ:

- ฟیلด์ **balance** เป็นตัวพิมพ์เล็ก → มองไม่เห็นจากนอกแพ็คเกจ
- เมธอด **SetBalance** และ **GetBalance** เป็นตัวพิมพ์ใหญ่ → เรียกใช้จากภายนอกได้
- การเปลี่ยนแปลงค่าผ่านเมธอดที่มี validation เท่านั้น

ตัวอย่างโค้ดจริง (Go)

```
// bank/account.go (แพ็คเกจ bank)
package bank

import "errors"

// Account คือ struct ที่มี encapsulation (การห่อหุ้มข้อมูล)
// ฟیلด์ทั้งหมดเป็น unexported (ขึ้นต้นด้วยตัวพิมพ์เล็ก) ทำให้เข้าถึงได้เฉพาะภายในแพ็คเกจ bank เท่านั้น
type Account struct {
    // unexported field (private) - ไม่สามารถเข้าถึงโดยตรงจากภายนอกแพ็คเกจ
    accountNumber string
    balance        float64
}

// Constructor (ฟังก์ชัน exported - ขึ้นต้นด้วยตัวพิมพ์ใหญ่)
// ใช้สำหรับสร้าง instance ของ Account อย่างถูกต้อง (เหมือนการสร้าง object ใน OOP)
func NewAccount(number string, initialBalance float64) *Account {
    // ตรวจสอบเงื่อนไขเบื้องต้น: ยอดเริ่มต้นต้องไม่ติดลบ
    if initialBalance < 0 {
        initialBalance = 0
    }
    // คืนค่า pointer ของ Account ที่สร้างขึ้นใหม่
    return &Account{
        accountNumber: number,
        balance:       initialBalance,
    }
}

// Exported method: Getter (อ่านค่าฟیلด์ balance)
// เนื่องจาก balance เป็น unexported จึงต้องมีเมธอดนี้เพื่อให้ภายนอกอ่านค่าได้
func (a *Account) GetBalance() float64 {
    return a.balance
}

// Exported method: Setter พร้อม validation
// ควบคุมการเปลี่ยนแปลงค่า balance โดยเพิ่มกฎทางธุรกิจ (ห้ามติดลบ)
```

```

func (a *Account) SetBalance(amount float64) error {
    if amount < 0 {
        return errors.New("ยอดเงินต้องไม่ติดลบ")
    }
    a.balance = amount
    return nil
}

// เมธอดอื่นๆ: ฝากเงิน (Deposit)
// มีการตรวจสอบจำนวนเงินก่อนดำเนินการ
func (a *Account) Deposit(amount float64) error {
    if amount <= 0 {
        return errors.New("จำนวนเงินฝากต้องมากกว่า 0")
    }
    a.balance += amount
    return nil
}

// เมธอดอื่นๆ: ถอนเงิน (Withdraw)
// ตรวจสอบทั้งจำนวนเงินที่ถอนและยอดคงเหลือเพียงพอหรือไม่
func (a *Account) Withdraw(amount float64) error {
    if amount <= 0 || amount > a.balance {
        return errors.New("จำนวนเงินไม่ถูกต้องหรือยอดไม่พอ")
    }
    a.balance -= amount
    return nil
}

```

```

// main.go (ต่างแพ็คเกจ - ไม่ใช่แพ็คเกจ bank)
package main

import (
    "fmt"
    "yourmodule/bank" // import แพ็คเกจ bank ที่เราสร้าง
)

func main() {
    // สร้าง instance ของ Account โดยใช้ Constructor (NewAccount)
    // ไม่สามารถสร้างโดยใช้ Account{} โดยตรงได้เพราะฟิลด์เป็น unexported
    acc := bank.NewAccount("123-456", 1000)

    // อ่านยอดเงินผ่าน Getter (เมธอด exported)
    fmt.Println("ยอดคงเหลือ:", acc.GetBalance()) // 1000

    // ฝากเงิน 500 บาท (ผ่านเมธอด Deposit ที่มี validation)
    acc.Deposit(500)
    fmt.Println("หลังฝาก:", acc.GetBalance()) // 1500

    // พยายามตั้งยอดเป็น -100 (ติดลบ) ผ่าน Setter
    // Setter จะตรวจสอบและปฏิเสธ พร้อมคืน error
}

```

```
err := acc.SetBalance(-100)
if err != nil {
    fmt.Println("Error:", err) // ผลลัพธ์: ยอดเงินต้องไม่ติดลบ
}

// บรรทัดต่อไปนี้จะคอมไพล์ไม่ผ่าน เพราะ balance เป็น unexported
// ไม่สามารถเข้าถึงโดยตรงจากแพ็คเกจ main ได้
// acc.balance = 2000 // ✗ compile error: balance unexported
}
```

🔒 อธิบายหลักการ Encapsulation ใน Go (จากตัวอย่าง)

1. การควบคุมการมองเห็น (Visibility) ด้วยตัวพิมพ์ใหญ่/เล็ก

- **Unexported (private)** : ฟیلด์ `accountNumber`, `balance` ขึ้นต้นด้วยตัวพิมพ์เล็ก → มองเห็นได้เฉพาะภายในแพ็คเกจ `bank` เท่านั้น
- **Exported (public)** : เมธอด `GetBalance`, `SetBalance`, `Deposit`, `Withdraw` ขึ้นต้นด้วยตัวพิมพ์ใหญ่ → มองเห็นได้จากทุกแพ็คเกจ (รวมถึง `main`)

2. การเข้าถึงข้อมูลทางอ้อมผ่านเมธอด (Getter/Setter)

- ภายนอกไม่สามารถแก้ไข `balance` ได้โดยตรง (`acc.balance = 2000 ✗`)
- ต้องใช้เมธอด `SetBalance(amount)` ซึ่งมีการตรวจสอบเงื่อนไข (`amount >= 0`) ก่อนเปลี่ยนค่า
- การอ่านค่าก็ต้องใช้ `GetBalance()` แทนการเข้าถึงโดยตรง

3. Validation (การตรวจสอบความถูกต้อง) ใน Setter และเมธอดอื่นๆ

- `SetBalance` : ห้ามตั้งยอดติดลบ
- `Deposit` : จำนวนเงินฝากต้องมากกว่า 0
- `Withdraw` : จำนวนเงินถอนต้องมากกว่า 0 และไม่เกินยอดคงเหลือ

4. Constructor (ฟังก์ชันสำหรับสร้าง object)

- Go ไม่มี constructor ในตัว แต่ใช้ฟังก์ชัน `NewAccount` ซึ่งเป็น exported (พิมพ์ใหญ่) เพื่อสร้าง instance
- ภายใน constructor สามารถตั้งค่าเริ่มต้นและตรวจสอบความถูกต้องได้ (เช่น ถ้า `initialBalance` ติดลบ ให้ตั้งเป็น 0)

5. ประโยชน์ของ Encapsulation ในตัวอย่างนี้

- **ป้องกันสถานะไม่ถูกต้อง** : ไม่มียอดคงเหลือติดลบเกิดขึ้นได้
- **ซ่อนรายละเอียดภายใน** : การเปลี่ยนแปลงฟیلด์ `balance` ต้องผ่านกฎทางธุรกิจเท่านั้น
- **ปรับเปลี่ยนง่าย** : ถ้าอยากเพิ่ม validation อื่น (เช่น ห้ามยอดเกิน 1,000,000) ก็แค่แก้ไขในเมธอด Setter โดยไม่ต้องแก้ไขโค้ดภายนอก

6. ความแตกต่างจากภาษา OOP แบบคลาสสิก (Java/C++)

- Go ใช้ **package** เป็นหน่วยของการ encapsulate (ไม่ใช่ class)
- ไม่มีคีย์เวิร์ด `private/public` แต่ใช้ **การตั้งชื่อ** แทน
- ไม่มี `this` โดยปริยาย แต่ใช้ **receiver** (`a *Account`) ซึ่งชัดเจนกว่า

ตัวอย่างการใช้งานจริงเพิ่มเติม

```
// สมมติว่ามีการเรียกใช้ในระบบธนาคาร
func Transfer(from, to *bank.Account, amount float64) error {
    if err := from.Withdraw(amount); err != nil {
        return err
    }
    if err := to.Deposit(amount); err != nil {
        // ถ้าโอนเข้า failed ให้คืนเงิน (Rollback)
        from.Deposit(amount)
        return err
    }
    return nil
}
```

จะเห็นว่า **Transfer** ไม่สามารถเข้าถึง **balance** โดยตรงได้ ต้องใช้เมธอดที่ปลอดภัยเท่านั้น → ช่วยป้องกันความผิดพลาดและรักษาความถูกต้องของข้อมูล

สรุป

Encapsulation ใน Go ทำได้โดย:

- **Unexported fields** (พิมพ์เล็ก) → ห่อหุ้มข้อมูลไว้ภายในแพ็คเกจ
- **Exported methods** (พิมพ์ใหญ่) → เปิดช่องทางเข้าถึงที่ปลอดภัย พร้อม validation
- ทำให้โค้ด ปลอดภัย, บำรุงรักษาง่าย และ ลดการพึ่งพากัน (low coupling)

2. Inheritance (การสืบทอด) ใน Go

Inheritance คืออะไรใน Go?

Go ไม่มีการสืบทอดแบบคลาส (class-based inheritance) แต่ใช้ **Composition** และ **Embedding** (การฝัง struct) เพื่อให้ได้ความสามารถคล้ายกัน โดย struct ลูกสามารถเข้าถึงฟิลด์และเมธอดของ struct แม่ได้เสมือน "is-a" บางส่วน

Inheritance มีกี่แบบใน Go?

Go มีรูปแบบการ "สืบทอด" ผ่าน Composition/Embedding ได้ 3 แบบ:

แบบ	คำอธิบาย	ตัวอย่าง
1. Struct Embedding	ฝัง struct แม่ลงใน struct ลูก (คล้าย single inheritance)	<code>type Dog struct { Animal }</code>
2. Multiple Embedding	ฝังหลาย struct (คล้าย multiple inheritance แต่ไม่มีความขัดแย้ง)	<code>type Bird struct { Animal, Flyer }</code>

แบบ	คำอธิบาย	ตัวอย่าง
3. Interface Embedding	ฝัง interface เพื่อสร้าง interface ใหม่	<code>type ReaderWriter interface { Reader; Writer }</code>

ข้อควรรู้: Go ไม่มี method overriding แบบอัตโนมัติ แต่เราสามารถ "override" ได้โดยการนิยามเมธอดชื่อเดียวกันใน struct ลูก (shadowing) และสามารถเรียกเมธอดของ struct แม่ผ่าน `Animal.Eat()` ได้

🔗 ใช้อย่างไร? ในกรณีไหน?

- **ใช้เมื่อ** ต้องการนำโค้ดมาใช้ซ้ำ (reuse) โดยไม่ต้องเขียนซ้ำ
- **กรณีใช้งานจริง:**
 - ระบบยานพาหนะ: `Vehicle` มี `Start()`, `Car` ฝัง `Vehicle` และเพิ่ม `OpenSunroof()`
 - ระบบ HTTP handler: ฝัง `http.ServeMux`
 - ระบบ Logger: ฝัง `log.Logger`

📄 แผนภาพ Dataflow (Flowchart TB) – Struct Embedding ใน Go

```

flowchart TB
    subgraph Base["struct แม่ (Animal)"]
        A["ฟิลด์: Name string<br>เมธอด: Eat(), Sleep()"]
    end

    subgraph Derived["struct ลูก (Dog)"]
        B["ฝัง Animal<br>type Dog struct { Animal }"]
        C["เพิ่มฟิลด์ Breed string<br>เพิ่มเมธอด Bark()"]
        D["ต้องการเปลี่ยนเมธอด Eat?"]
        E["กำหนด func (d Dog) Eat() {<br>    fmt.Println(d.Name, \"กินกระดูก\")<br>}"]
    end

    F["d := Dog{Animal{Name: \"ทองคำ\"}, \"โกลเด้น\"}<br>d.Eat()"]
    G["เรียกเมธอดของ Animal ที่ถูกฝัง<br>d.Animal.Sleep()"]

    A -- ฝังเข้า --> B
    B --> C
    C --> D
    D -- ใช่ --> E
    D -- ไม่ใช่ --> C
    E --> F
    F -. -> G
  
```

คำอธิบายแผนภาพ:

- `Dog` ฝัง `Animal` → สามารถเรียก `d.Name`, `d.Eat()` ได้โดยตรง
- ถ้า `Dog` ไม่มีเมธอด `Eat()` จะใช้ของ `Animal` แทน
- ถ้าต้องการ "override" ก็แค่เขียนเมธอด `Eat()` ใน `Dog`

- การเข้าถึงเมธอดของแม่ทำได้โดย `d.Animal.Eat()`

ตัวอย่างโค้ดจริง (Go)

```
package main

import "fmt"

// ===== struct แม่ (Base) =====
type Animal struct {
    Name string
}

func (a Animal) Eat() {
    fmt.Printf("%s กำลังกินอาหาร\n", a.Name)
}

func (a Animal) Sleep() {
    fmt.Printf("%s กำลังนอน\n", a.Name)
}

// ===== struct ลูก ใช้ Embedding =====
type Dog struct {
    Animal // ผีง struct Animal (ไม่มีชื่อฟิลด์)
    Breed string
}

// "Override" เมธอด Eat (shadowing)
func (d Dog) Eat() {
    fmt.Printf("%s (พันธุ์%s) กำลังกินกระดูก\n", d.Name, d.Breed)
}

// เมธอดเพิ่มเติมเฉพาะ Dog
func (d Dog) Bark() {
    fmt.Printf("%s เห่า โห้งๆ\n", d.Name)
}

// ===== Multiple Embedding =====
type Flyer struct {
    MaxSpeed int
}

func (f Flyer) Fly() {
    fmt.Printf("บินได้ด้วยความเร็วสูงสุด %d km/h\n", f.MaxSpeed)
}

// Bird ผีงทั้ง Animal และ Flyer
type Bird struct {
    Animal
    Flyer
}
```

```

    WingSpan float64
}

// ===== การใช้งานจริง =====
func main() {
    // Single embedding
    dog := Dog{
        Animal: Animal{Name: "ทองคำ"},
        Breed:   "โกลเด้น",
    }

    dog.Eat()    // ทองคำ (พันธุ์โกลเด้น) กำลังกินกระดูก (ใช้ของ Dog)
    dog.Sleep()  // ทองคำ กำลังนอน (ใช้ของ Animal เพราะ Dog ไม่มี Sleep)
    dog.Bark()    // ทองคำ เห่า โห้งๆ

    // เรียกเมธอดของ Animal โดยตรง (ถ้าถูก shadow)
    dog.Animal.Eat() // ทองคำ กำลังกินอาหาร

    // Multiple embedding
    bird := Bird{
        Animal: Animal{Name: "นกอินทรี"},
        Flyer:   Flyer{MaxSpeed: 160},
        WingSpan: 2.5,
    }
    bird.Eat() // นกอินทรี กำลังกินอาหาร
    bird.Fly() // บินได้ด้วยความเร็วสูงสุด 160 km/h
}

```

🐶 3. Polymorphism (พหุสัณฐาน) ใน Go

🔍 Polymorphism คืออะไรใน Go?

Polymorphism ใน Go ทำผ่าน **Interface** เป็นหลัก โดย interface กำหนดชุดของเมธอดที่ประเภทต่างๆ (struct) ต้อง implement จากนั้นตัวแปรชนิด interface สามารถเก็บค่าได้หลายประเภท และเรียกเมธอดเดียวกันโดยได้พฤติกรรมต่างกันตามประเภทจริง (runtime polymorphism)

นอกจากนี้ยังมี **compile-time polymorphism** ผ่าน **Generic functions** (ตั้งแต่ Go 1.18)

📊 Polymorphism มีกี่แบบใน Go?

แบ่งเป็น 2 แบบ:

แบบ	เวลาทำงาน	กลไก	ตัวอย่าง
1. Runtime Polymorphism	ตอน run	Interface + Dynamic dispatch	<code>var a Animal; a = Dog{}; a.Speak()</code>

แบบ	เวลา ทำงาน	กลไก	ตัวอย่าง
2. Compile-time Polymorphism	ตอน compile	Generics (type parameters)	<code>func Print[T any](val T)</code>

หมายเหตุ: Method Overloading (ชื่อเมธอดเดียวกันแต่พารามิเตอร์ต่างกัน) **ไม่มีใน Go** – ใช้วิธีตั้งชื่อต่างกันหรือใช้ variadic parameters แทน

🔗 ใช้อย่างไร? ในกรณีไหน?

- **ใช้เมื่อ** ต้องการเขียนโค้ดที่ยืดหยุ่น รับค่าได้หลายประเภทผ่าน interface เดียว
- **กรณีใช้งานจริง:**
 - ระบบการจ่ายเงิน: `PaymentMethod` interface มี `Pay(amount)` – `CreditCard`, `PayPal`, `Cash`
 - ระบบ Logging: `Logger` interface – `FileLogger`, `ConsoleLogger`, `RemoteLogger`
 - HTTP handlers: `http.Handler` interface

📊 แผนภาพ Dataflow (Flowchart TB) – Polymorphism ใน Go

```

flowchart TB
    subgraph Interface["Interface (Shape)"]
        A["type Shape interface {<br>    Area() float64<br>}"]
    end

    subgraph Implement["Structs ที่ implement Shape"]
        B1["type Circle struct { Radius float64 }<br>func (c Circle) Area() float64 { return π*r² }"]
        B2["type Rectangle struct { W, H float64 }<br>func (r Rectangle) Area() float64 { return w*h }"]
    end

    subgraph Runtime["ขณะรันโปรแกรม"]
        C["var s Shape"]
        D["s เก็บ object ประเภทใด?"]
        E1["s = Circle{5} → s.Area() = π*25"]
        E2["s = Rectangle{3,4} → s.Area() = 12"]
    end

    A -- implement โดย --> B1
    A -- implement โดย --> B2
    C --> D
    D -- ถ้าเป็น Circle --> E1
    D -- ถ้าเป็น Rectangle --> E2

```

คำอธิบายแผนภาพ:

- `Shape` interface กำหนดเมธอด `Area()`

- Circle และ Rectangle implement Area() ตามรูปแบบของตน
- ตัวแปร s ชนิด Shape สามารถชี้ไปที่ Circle หรือ Rectangle ก็ได้
- เมื่อเรียก s.Area() Go จะเรียกเมธอดของประเภทจริงโดยอัตโนมัติ

ตัวอย่างโค้ดจริง (Go)

```
package main

import (
    "fmt"
    "math"
)

// ===== Runtime Polymorphism ผ่าน Interface =====
// 1. ประกาศ interface
type Shape interface {
    Area() float64
    Perimeter() float64
}

// 2. Struct ที่ implement Shape
type Circle struct {
    Radius float64
}

func (c Circle) Area() float64 {
    return math.Pi * c.Radius * c.Radius
}

func (c Circle) Perimeter() float64 {
    return 2 * math.Pi * c.Radius
}

type Rectangle struct {
    Width, Height float64
}

func (r Rectangle) Area() float64 {
    return r.Width * r.Height
}

func (r Rectangle) Perimeter() float64 {
    return 2 * (r.Width + r.Height)
}

// ฟังก์ชันที่ใช้ Polymorphism (รับ interface)
func PrintShapeInfo(s Shape) {
    fmt.Printf("พื้นที่: %.2f, เส้นรอบรูป: %.2f\n", s.Area(), s.Perimeter())
}
```

```
// ===== Compile-time Polymorphism ผ่าน Generics (Go 1.18+) =====
func Sum[T int | float64](a, b T) T {
    return a + b
}

// Generic function กับ slice
func PrintSlice[T any](s []T) {
    for _, v := range s {
        fmt.Print(v, " ")
    }
    fmt.Println()
}

// ===== การใช้งานจริง =====
func main() {
    // Runtime Polymorphism
    var s Shape

    s = Circle{Radius: 5}
    fmt.Println("Circle:")
    fmt.Printf("Area: %.2f\n", s.Area()) // 78.54
    fmt.Printf("Perimeter: %.2f\n", s.Perimeter()) // 31.42
    PrintShapeInfo(s) // พื้นที่: 78.54, เส้นรอบรูป: 31.42

    s = Rectangle{Width: 4, Height: 6}
    fmt.Println("\nRectangle:")
    fmt.Printf("Area: %.2f\n", s.Area()) // 24.00
    PrintShapeInfo(s) // พื้นที่: 24.00, เส้นรอบรูป: 20.00

    // ใช้ slice ของ interface
    shapes := []Shape{
        Circle{Radius: 2},
        Rectangle{Width: 3, Height: 4},
        Circle{Radius: 7},
    }

    fmt.Println("\nAll shapes:")
    for _, sh := range shapes {
        fmt.Printf("- Area = %.2f\n", sh.Area())
    }

    // Compile-time Polymorphism (Generics)
    fmt.Println("\nGenerics:")
    fmt.Println(Sum(10, 20)) // 30 (int)
    fmt.Println(Sum(3.5, 2.7)) // 6.2 (float64)

    PrintSlice([]int{1, 2, 3})
    PrintSlice([]string{"a", "b", "c"})
}
```

🔗 สรุปเปรียบเทียบหลักการ OOP ใน Go เทียบกับภาษา OOP ดั้งเดิม

หลักการ	ใน Java/C++	ใน Go
Encapsulation	<code>private</code> , <code>protected</code> , <code>public</code>	ตัวพิมพ์เล็ก (unexported) vs ตัวพิมพ์ใหญ่ (exported)
Inheritance	<code>extends</code> (class-based)	Struct embedding (composition แทน)
Polymorphism	Method overloading + overriding + interface	Interface (runtime) + Generics (compile-time)
คอนสตรักเตอร์	Constructor ชื่อเดียวกับคลาส	ฟังก์ชัน <code>NewXxx()</code> (convention)
this/super	<code>this</code> , <code>super</code>	รับเป็น receiver (<code>func (t T) method()</code>)

🔗 ข้อควรจำสำหรับ Go

1. ไม่มี `class` → ใช้ `struct` และ `interface`
2. ไม่มี `inheritance` แบบ `is-a` → ใช้ `embedding` เพื่อ reuse โค้ด (ควรใช้ `composition` มากกว่า)
3. การควบคุมการเข้าถึง → ดูที่ ตัวอักษรตัวแรก (พิมพ์ใหญ่/เล็ก)
4. **Polymorphism** → **interface** เป็นหัวใจสำคัญ (implement โดยนัย ไม่ต้องประกาศ `implements`)
5. **Method overloading** ไม่มี → ใช้ชื่อเมธอดต่างกัน หรือใช้ variadic/empty interface ร่วมกับ `type switch`

```
// ตัวอย่าง: Polymorphism ที่เป็นเอกลักษณ์ของ Go (empty interface + type switch)
func Describe(v interface{}) {
    switch val := v.(type) {
    case int:
        fmt.Printf("int: %d\n", val)
    case string:
        fmt.Printf("string: %s\n", val)
    default:
        fmt.Printf("unknown type: %T\n", val)
    }
}
```

ตัวอย่างเพิ่มเติมในบริบทเฉพาะ (Web API, การทดสอบ unit test ด้วย interface)

🚀 ตัวอย่างเพิ่มเติม: Web API + Unit Testing ด้วย Interface ใน Go

ต่อไปนี้เป็นตัวอย่างการนำหลักการทั้งสาม (Encapsulation, Inheritance/Embedding, Polymorphism) มาใช้ใน **Web API (REST API)** และ **Unit Testing** โดยใช้ **Interface** เพื่อให้ทดสอบได้ง่าย (Dependency Injection + Mocking)

📁 โครงสร้างโปรเจกต์ตัวอย่าง

```

userapi/
├── go.mod
├── main.go
├── internal/
│   ├── domain/
│   │   ├── user.go          # struct และ interface
│   │   └── repository/
│   │       ├── user_repo.go    # interface + real implementation
│   │       └── mock_user_repo.go # mock สำหรับ test
│   ├── service/
│   │   └── user_service.go    # service layer
│   └── handler/
│       └── user_handler.go    # HTTP handler
└── service_test.go          # unit test

```

1 Encapsulation ใน Web API (domain/user.go)

```

// internal/domain/user.go
package domain

import "errors"

// User struct: ฟิลด์เป็น unexported (encapsulation)
type User struct {
    id      int
    username string
    email   string
    isActive bool
}

// Constructor (exported)
func NewUser(id int, username, email string) (*User, error) {
    if username == "" || email == "" {
        return nil, errors.New("username และ email ห้ามว่าง")
    }
    return &User{
        id:      id,
        username: username,
        email:   email,
        isActive: true,
    }, nil
}

// Getters (อ่านค่าได้อย่างเดียว)
func (u *User) ID() int      { return u.id }
func (u *User) Username() string { return u.username }
func (u *User) Email() string { return u.email }
func (u *User) IsActive() bool { return u.isActive }

```

```
// Setter พร้อม validation (encapsulation ควบคุมการแก้ไข)
func (u *User) Deactivate() {
    u.isActive = false
}

func (u *User) UpdateEmail(newEmail string) error {
    if newEmail == "" {
        return errors.New("email ห้ามว่าง")
    }
    u.email = newEmail
    return nil
}
```

☑ Encapsulation ใน API:

- ฟังก์ชัน `id`, `username` ไม่สามารถแก้ไขได้โดยตรงจาก handler
- การแก้ไขต้องผ่านเมธอดที่มี validation (`UpdateEmail`, `Deactivate`)

2 Inheritance (Struct Embedding) – reusable base model

```
// internal/domain/base.go (ฝังใน repository หรือ service)
package domain

import "time"

// BaseModel ฝังใน struct อื่นเพื่อ reuse ฟังก์ชันทั่วไป
type BaseModel struct {
    CreatedAt time.Time
    UpdatedAt time.Time
}

func (b *BaseModel) SetTimestamps() {
    now := time.Now()
    if b.CreatedAt.IsZero() {
        b.CreatedAt = now
    }
    b.UpdatedAt = now
}

// ตัวอย่างฝังใน UserEntity (สำหรับ database layer)
type UserEntity struct {
    BaseModel // embedding
    ID         int
    Username   string
    Email      string
    IsActive   bool
}
```

☑ Inheritance ใน API:

- `UserEntity` มีฟิลด์ `CreatedAt`, `UpdatedAt` และเมธอด `SetTimestamps()` โดยอัตโนมัติ
- ช่วยลดโค้ดซ้ำในหลาย struct (เช่น `Product`, `Order`)

3 Polymorphism ผ่าน Interface – เพื่อการทดสอบ (Testable API)

◊ ขั้นที่ 1: กำหนด interface ของ Repository

```
// internal/repository/user_repo.go
package repository

import (
    "context"
    "yourmodule/internal/domain"
)

// UserRepository interface (polymorphism)
type UserRepository interface {
    GetByID(ctx context.Context, id int) (*domain.User, error)
    Save(ctx context.Context, user *domain.User) error
}
```

◊ ขั้นที่ 2: Implement จริง (ใช้ database)

```
// internal/repository/user_repo.go (ต่อ)
type userRepositoryImpl struct {
    // db *sql.DB หรือ gorm.DB
}

func NewUserRepository() UserRepository {
    return &userRepositoryImpl{}
}

func (r *userRepositoryImpl) GetByID(ctx context.Context, id int)
(*domain.User, error) {
    // จำลองการดึงจาก DB
    // ในของจริง: row := db.QueryRow("SELECT ...")
    return domain.NewUser(id, "john_doe", "john@example.com")
}

func (r *userRepositoryImpl) Save(ctx context.Context, user *domain.User) error
{
    // INSERT หรือ UPDATE
    return nil
}
```

◇ ชั้นที่ 3: Service layer ที่ใช้ interface (Polymorphism)

```
// internal/service/user_service.go
package service

import (
    "context"
    "errors"
    "yourmodule/internal/domain"
    "yourmodule/internal/repository"
)

type UserService struct {
    repo repository.UserRepository // รับ interface ไม่ใช่ concrete type
}

// Constructor รับ interface (Dependency Injection)
func NewUserService(repo repository.UserRepository) *UserService {
    return &UserService{repo: repo}
}

func (s *UserService) GetUser(ctx context.Context, id int) (*domain.User, error) {
    if id <= 0 {
        return nil, errors.New("invalid user id")
    }
    return s.repo.GetByID(ctx, id)
}

func (s *UserService) DeactivateUser(ctx context.Context, id int) error {
    user, err := s.repo.GetByID(ctx, id)
    if err != nil {
        return err
    }
    user.Deactivate() // เรียกเมธอด encapsulated
    return s.repo.Save(ctx, user)
}
```

◇ ชั้นที่ 4: HTTP Handler (ใช้ Service)

```
// internal/handler/user_handler.go
package handler

import (
    "encoding/json"
    "net/http"
    "strconv"
    "yourmodule/internal/service"
)
```

```

type UserHandler struct {
    userService *service.UserService
}

func NewUserHandler(svc *service.UserService) *UserHandler {
    return &UserHandler{userService: svc}
}

func (h *UserHandler) GetUser(w http.ResponseWriter, r *http.Request) {
    idStr := r.URL.Query().Get("id")
    id, _ := strconv.Atoi(idStr)

    user, err := h.userService.GetUser(r.Context(), id)
    if err != nil {
        http.Error(w, err.Error(), http.StatusNotFound)
        return
    }

    // ใช้ getter (encapsulation) สร้าง response
    resp := map[string]interface{}{
        "id":      user.ID(),
        "username": user.Username(),
        "email":   user.Email(),
        "active":  user.IsActive(),
    }
    json.NewEncoder(w).Encode(resp)
}

```

◊ ขั้นที่ 5: main.go – ประกอบ Dependency

```

// main.go
package main

import (
    "log"
    "net/http"
    "yourmodule/internal/handler"
    "yourmodule/internal/repository"
    "yourmodule/internal/service"
)

func main() {
    // real repository
    userRepo := repository.NewUserRepository()
    userSvc := service.NewUserService(userRepo)
    userHandler := handler.NewUserHandler(userSvc)

    http.HandleFunc("/user", userHandler.GetUser)
}

```

```
log.Fatal(http.ListenAndServe(":8080", nil))
}
```

Unit Testing ด้วย Mock Interface (Polymorphism ช่วยให้ทดสอบได้)

เราสร้าง **Mock** ที่ implement **UserRepository** เดียวกัน แล้ว inject เข้า service

สร้าง Mock (manual หรือใช้ testify/mock)

```
// internal/repository/mock_user_repo.go
package repository

import (
    "context"
    "yourmodule/internal/domain"
)

type MockUserRepository struct {
    GetByIDFunc func(ctx context.Context, id int) (*domain.User, error)
    SaveFunc     func(ctx context.Context, user *domain.User) error
}

func (m *MockUserRepository) GetByID(ctx context.Context, id int)
(*domain.User, error) {
    if m.GetByIDFunc != nil {
        return m.GetByIDFunc(ctx, id)
    }
    return nil, nil
}

func (m *MockUserRepository) Save(ctx context.Context, user *domain.User) error
{
    if m.SaveFunc != nil {
        return m.SaveFunc(ctx, user)
    }
    return nil
}
```

เขียน Unit Test (service_test.go)

```
// service_test.go
package service

import (
    "context"
    "errors"
    "testing"
```

```

    "yourmodule/internal/domain"
    "yourmodule/internal/repository"
)

func TestUserService_GetUser_Success(t *testing.T) {
    // 1. สร้าง mock
    mockRepo := &repository.MockUserRepository{
        GetByIDFunc: func(ctx context.Context, id int) (*domain.User, error) {
            // จำลองการคืน user
            return domain.NewUser(id, "testuser", "test@example.com")
        },
    }

    // 2. inject mock เข้า service
    svc := NewUserService(mockRepo)

    // 3. เรียกใช้
    user, err := svc.GetUser(context.Background(), 123)

    // 4. ตรวจสอบผลลัพธ์
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }
    if user.ID() != 123 {
        t.Errorf("expected ID 123, got %d", user.ID())
    }
    if user.Username() != "testuser" {
        t.Errorf("expected username testuser, got %s", user.Username())
    }
}

func TestUserService_GetUser_InvalidID(t *testing.T) {
    mockRepo := &repository.MockUserRepository{}
    svc := NewUserService(mockRepo)

    _, err := svc.GetUser(context.Background(), -5)
    if err == nil {
        t.Error("expected error for invalid id, got nil")
    }
}

func TestUserService_DeactivateUser_CallsSave(t *testing.T) {
    saved := false
    mockRepo := &repository.MockUserRepository{
        GetByIDFunc: func(ctx context.Context, id int) (*domain.User, error) {
            return domain.NewUser(id, "active", "a@b.com")
        },
        SaveFunc: func(ctx context.Context, user *domain.User) error {
            saved = true
            if user.IsActive() {
                t.Error("user should be deactivated before save")
            }
            return nil
        },
    }
}

```

```

    },
}

svc := NewUserService(mockRepo)
err := svc.DeactivateUser(context.Background(), 1)
if err != nil {
    t.Fatalf("unexpected error: %v", err)
}
if !saved {
    t.Error("expected Save to be called")
}
}

```

ใช้ testify/mock (สะดวกขึ้น)

```

// go get github.com/stretchr/testify/mock

// internal/repository/mock_user_repo_testify.go
package repository

import (
    "context"
    "github.com/stretchr/testify/mock"
    "yourmodule/internal/domain"
)

type MockUserRepoTestify struct {
    mock.Mock
}

func (m *MockUserRepoTestify) GetByID(ctx context.Context, id int)
(*domain.User, error) {
    args := m.Called(ctx, id)
    if args.Get(0) == nil {
        return nil, args.Error(1)
    }
    return args.Get(0).(*domain.User), args.Error(1)
}

func (m *MockUserRepoTestify) Save(ctx context.Context, user *domain.User)
error {
    args := m.Called(ctx, user)
    return args.Error(0)
}

// ใช้ใน test
func TestWithTestify(t *testing.T) {
    mockRepo := new(repository.MockUserRepoTestify)
    mockRepo.On("GetByID", mock.Anything, 100).Return(domain.NewUser(100,
    "john", "j@k.com"))
}

```

```

mockRepo.On("Save", mock.Anything, mock.Anything).Return(nil)

svc := NewUserService(mockRepo)
user, _ := svc.GetUser(context.Background(), 100)
if user.Username() != "john" {
    t.Error("fail")
}
mockRepo.AssertExpectations(t)
}

```

แผนภาพ Dataflow ของ Web API + Unit Test (Flowchart TB)

```

flowchart TB
    subgraph Production["โหมตจริง (Production)"]
        Client[HTTP Client] --> Handler[UserHandler]
        Handler --> Service[UserService]
        Service --> RealRepo["RealRepository<br>(implement interface)"]
        RealRepo --> DB[(Database)]
    end

    subgraph Test["โหมตทดสอบ (Unit Test)"]
        TestFunc[Test Function] --> Service2[UserService เดียวกัน]
        Service2 --> MockRepo["MockUserRepository<br>(implement interface เดียว)"]
        MockRepo --> Assertion[ตรวจสอบผลลัพธ์<br>+ ตรวจสอบว่า mock ถูกเรียก]
    end

    subgraph InterfaceDef["Interface ที่เป็นหัวใจ"]
        Interface["UserRepository interface<br>{ GetByID, Save }"]
    end

    RealRepo -. implement .-> Interface
    MockRepo -. implement .-> Interface
    Service --> Interface
    Service2 --> Interface

```

อธิบาย:

- **UserService** ทำงานกับ **UserRepository interface** ไม่รู้ว่าข้างหลังเป็นของจริงหรือ mock
- ใน production ส่ง **RealRepository**
- ใน test ส่ง **MockRepository** ทำให้ test เร็วและไม่ต้องพึ่ง database จริง
- นี่คือพลังของ **Polymorphism ผ่าน Interface** ช่วยให้โค้ด **testable**

☒ สรุปประโยชน์ของ Interface ในการทดสอบ Web API

สิ่งที่ได้

คำอธิบาย

สิ่งที่ได้	คำอธิบาย
Decoupling	Handler/Service ไม่ผูกติดกับ DB เฉพาะ สามารถเปลี่ยน implementation ได้
Mocking	สร้าง Mock ที่ implement interface เดิม แล้ว inject ผ่าน constructor
Test Coverage	ทดสอบ business logic ได้โดยไม่ต้องรัน database
Parallel Test	mock ไม่มี shared state ทำให้ test รันขนานกันได้
Fast	ไม่มี I/O จริง (network, disk)

ตัวอย่างคำสั่งรัน

```
# รัน web server
go run main.go

# ทดสอบ unit test
go test -v ./...

# ทดสอบ API (อีก terminal)
curl "http://localhost:8080/user?id=1"
```

ข้อคิดสำหรับ Go Web API

1. **Always accept interfaces, return structs** – รับ interface ในฟังก์ชัน/constructor
2. **Keep interfaces small** – 1-2 เมธอดต่อ interface (เช่น `UserRepository`)
3. **Use embedding** สำหรับ middleware หรือ base handler
4. **Encapsulation** ใช้ unexported fields + exported methods เพื่อควบคุม state
5. **Test with interfaces** – สร้าง mock ได้ง่ายมาก

ตัวอย่างเพิ่มเติม การ test HTTP handler โดยใช้ `httptest` หรือการ test middleware

การทดสอบ HTTP Handler และ Middleware ใน Go ด้วย `httptest`

ต่อไปนี้เป็นตัวอย่างการทดสอบ HTTP handler และ middleware โดยใช้แพ็คเกจ `net/http/httptest` ควบคู่กับหลักการ **Dependency Injection** ผ่าน **interface** เพื่อให้ทดสอบได้ง่าย (Unit Test + Integration Test)

1. การทดสอบ HTTP Handler แบบหน่วย (Unit Test)

- ◊ โครงสร้าง Handler ที่รองรับการทดสอบ (ใช้ Interface)

```
// internal/service/user_service.go (สมมติ interface)
package service

import "context"

type UserService interface {
    GetUser(ctx context.Context, id int) (*User, error)
}

type User struct {
    ID    int
    Name  string
}

// internal/handler/user_handler.go
package handler

import (
    "encoding/json"
    "net/http"
    "strconv"
    "yourmodule/internal/service"
)

type UserHandler struct {
    userService service.UserService // รับ interface
}

func NewUserHandler(svc service.UserService) *UserHandler {
    return &UserHandler{userService: svc}
}

func (h *UserHandler) GetUser(w http.ResponseWriter, r *http.Request) {
    idStr := r.URL.Query().Get("id")
    id, err := strconv.Atoi(idStr)
    if err != nil || id <= 0 {
        http.Error(w, "invalid id", http.StatusBadRequest)
        return
    }

    user, err := h.userService.GetUser(r.Context(), id)
    if err != nil {
        http.Error(w, err.Error(), http.StatusNotFound)
        return
    }

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(user)
}
```

◇ สร้าง Mock Service (implement interface)

```
// internal/service/mock_user_service.go
package service

import "context"

type MockUserService struct {
    GetUserFunc func(ctx context.Context, id int) (*User, error)
}

func (m *MockUserService) GetUser(ctx context.Context, id int) (*User, error) {
    if m.GetUserFunc != nil {
        return m.GetUserFunc(ctx, id)
    }
    return nil, nil
}
```

◊ เขียน Unit Test โดยใช้ httptest

```
// handler/user_handler_test.go
package handler

import (
    "encoding/json"
    "net/http"
    "net/http/httptest"
    "testing"
    "yourmodule/internal/service"
)

func TestUserHandler_GetUser_Success(t *testing.T) {
    // 1. สร้าง mock service
    mockSvc := &service.MockUserService{
        GetUserFunc: func(ctx context.Context, id int) (*service.User, error) {
            return &service.User{ID: id, Name: "John Doe"}, nil
        },
    }

    // 2. สร้าง handler พร้อม inject mock
    handler := NewUserHandler(mockSvc)

    // 3. สร้าง request และ response recorder
    req := httptest.NewRequest(http.MethodGet, "/user?id=123", nil)
    rr := httptest.NewRecorder()

    // 4. เรียก handler
    handler.GetUser(rr, req)

    // 5. ตรวจสอบผลลัพธ์
    if status := rr.Code; status != http.StatusOK {
```

```

        t.Errorf("expected status OK, got %v", status)
    }

    var user service.User
    if err := json.NewDecoder(rr.Body).Decode(&user); err != nil {
        t.Fatal("failed to decode response")
    }
    if user.ID != 123 || user.Name != "John Doe" {
        t.Errorf("unexpected user: %+v", user)
    }
}

func TestUserHandler_GetUser_InvalidID(t *testing.T) {
    mockSvc := &service.MockUserService{}
    handler := NewUserHandler(mockSvc)

    req := httptest.NewRequest(http.MethodGet, "/user?id=invalid", nil)
    rr := httptest.NewRecorder()

    handler.GetUser(rr, req)

    if rr.Code != http.StatusBadRequest {
        t.Errorf("expected BadRequest, got %v", rr.Code)
    }
}

func TestUserHandler_GetUser_NotFound(t *testing.T) {
    mockSvc := &service.MockUserService{
        GetUserFunc: func(ctx context.Context, id int) (*service.User, error) {
            return nil, errors.New("user not found")
        },
    }
    handler := NewUserHandler(mockSvc)

    req := httptest.NewRequest(http.MethodGet, "/user?id=999", nil)
    rr := httptest.NewRecorder()

    handler.GetUser(rr, req)

    if rr.Code != http.StatusNotFound {
        t.Errorf("expected NotFound, got %v", rr.Code)
    }
}

```

🔗 2. การทดสอบ Middleware

- ◇ ตัวอย่าง Middleware: Logging และ Recovery

```
// internal/middleware/logging.go
package middleware

import (
    "log"
    "net/http"
    "time"
)

func LoggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        // ใช้ ResponseRecorder แบบกำหนดเอง เพื่อ capture status code
        rr := &responseRecorder{ResponseWriter: w, statusCode: http.StatusOK}
        next.ServeHTTP(rr, r)
        log.Printf("%s %s %d %v", r.Method, r.URL.Path, rr.statusCode,
time.Since(start))
    })
}

type responseRecorder struct {
    http.ResponseWriter
    statusCode int
}

func (rr *responseRecorder) WriteHeader(code int) {
    rr.statusCode = code
    rr.ResponseWriter.WriteHeader(code)
}
```

```
// internal/middleware/recovery.go
package middleware

import (
    "log"
    "net/http"
)

func RecoveryMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if err := recover(); err != nil {
                log.Printf("panic recovered: %v", err)
                http.Error(w, "Internal Server Error",
http.StatusInternalServerError)
            }
        }()
        next.ServeHTTP(w, r)
    })
}
```

◇ ทดสอบ Middleware ด้วย `httptest`

```
// middleware/logging_test.go
package middleware

import (
    "net/http"
    "net/http/httptest"
    "testing"
)

func TestLoggingMiddleware(t *testing.T) {
    // สร้าง handler ปลายทาง (dummy)
    finalHandler := http.HandlerFunc(func(w http.ResponseWriter, r
    *http.Request) {
        w.WriteHeader(http.StatusOK)
        w.Write([]byte("ok"))
    })

    // ห่อด้วย middleware
    handler := LoggingMiddleware(finalHandler)

    // สร้าง request
    req := httptest.NewRequest(http.MethodGet, "/test", nil)
    rr := httptest.NewRecorder()

    // เรียก middleware
    handler.ServeHTTP(rr, req)

    // ตรวจสอบ response
    if rr.Code != http.StatusOK {
        t.Errorf("expected status OK, got %d", rr.Code)
    }
    if rr.Body.String() != "ok" {
        t.Errorf("expected body 'ok', got %q", rr.Body.String())
    }
}
```

◇ ทดสอบ Middleware ที่มีการ panic

```
func TestRecoveryMiddleware_HandlesPanic(t *testing.T) {
    // handler ที่ panic
    panicHandler := http.HandlerFunc(func(w http.ResponseWriter, r
    *http.Request) {
        panic("something went wrong")
    })
}
```

```

handler := RecoveryMiddleware(panicHandler)

req := httptest.NewRequest(http.MethodGet, "/panic", nil)
rr := httptest.NewRecorder()

// ควรไม่ทำให้ test panic
handler.ServeHTTP(rr, req)

if rr.Code != http.StatusInternalServerError {
    t.Errorf("expected 500, got %d", rr.Code)
}
}

```

◇ ทดสอบ chain ของ middleware (หลายตัว)

```

func TestMiddlewareChain(t *testing.T) {
    // สร้าง handler ที่บันทึกการเรียก
    var called bool
    finalHandler := http.HandlerFunc(func(w http.ResponseWriter, r
    *http.Request) {
        called = true
        w.WriteHeader(http.StatusTeapot)
    })

    // ต่อ middleware: recovery -> logging -> final
    handler := RecoveryMiddleware(LoggingMiddleware(finalHandler))

    req := httptest.NewRequest(http.MethodGet, "/chain", nil)
    rr := httptest.NewRecorder()

    handler.ServeHTTP(rr, req)

    if !called {
        t.Error("final handler not called")
    }
    if rr.Code != http.StatusTeapot {
        t.Errorf("expected 418, got %d", rr.Code)
    }
}

```

แผนภาพ Flowchart การทดสอบ HTTP Handler และ Middleware

```

flowchart TB
    subgraph TestFunc["ฟังก์ชันทดสอบ (TestXXX)"]
        A1[สร้าง Mock Service / Mock Dependency]
        A2[สร้าง Handler พร้อม Inject Mock]
        A3[สร้าง httptest.NewRequest]
    end

```

```

    A4[สร้าง httptest.NewRecorder]
    A5[เรียก handler.ServeHTTP หรือ handler โดยตรง]
end

subgraph HandlerExecution["การทำงานของ Handler (ใน test)"]
    B1[Handler รับ request]
    B2{เรียก service interface}
    B3[Mock Service คืนค่าตามที่กำหนด]
    B4[Handler สร้าง response]
end

subgraph Assertions["ตรวจสอบผลลัพธ์"]
    C1[ตรวจสอบ Status Code]
    C2[ตรวจสอบ Response Body]
    C3[ตรวจสอบ Header]
    C4[ตรวจสอบว่า Mock ถูกเรียกตามที่คาด]
end

A1 --> A2 --> A3 --> A4 --> A5
A5 --> B1 --> B2 --> B3 --> B4 --> C1
C1 --> C2 --> C3 --> C4

```

แผนภาพการทดสอบ Middleware แบบ chain

```

flowchart LR
    Request[HTTP Request] --> MW1[Middleware 1<br>Logging]
    MW1 --> MW2[Middleware 2<br>Recovery]
    MW2 --> Handler[Final Handler]
    Handler --> Response[Response]

    subgraph Test["การจำลองใน httptest"]
        TestReq[httptest.NewRequest] --> CallHandler[handler.ServeHTTP]
        CallHandler --> Capture[httptest.ResponseRecorder]
        Capture --> Assert[ตรวจสอบ status/body]
    end

    MW1 -.-> Test
    MW2 -.-> Test

```

🔑 3. ตัวอย่างการทดสอบทั้งระบบ (Integration Test) ด้วย `httptest.Server`

บางครั้งเราต้องการทดสอบทั้ง routing และ middleware chain แบบ end-to-end โดยไม่ต้องเปิดพอร์ตจริง

```

// integration_test.go
package integration

import (

```

```

        "io"
        "net/http"
        "net/http/httpptest"
        "testing"
        "yourmodule/internal/handler"
        "yourmodule/internal/middleware"
        "yourmodule/internal/service"
    )

func TestIntegration_GetUserFlow(t *testing.T) {
    // 1. สร้าง service จริงหรือ mock ก็ได้ (ที่นี่ใช้ mock)
    mockSvc := &service.MockUserService{
        GetUserFunc: func(ctx context.Context, id int) (*service.User, error) {
            return &service.User{ID: id, Name: "Integration User"}, nil
        },
    }

    // 2. สร้าง handler
    userHandler := handler.NewUserHandler(mockSvc)

    // 3. สร้าง router พร้อม middleware chain
    mux := http.NewServeMux()
    mux.HandleFunc("/user", userHandler.GetUser)

    // ใส่ middleware ทัวไป
    var handler http.Handler = mux
    handler = middleware.LoggingMiddleware(handler)
    handler = middleware.RecoveryMiddleware(handler)

    // 4. สร้าง test server
    ts := httpptest.NewServer(handler)
    defer ts.Close()

    // 5. ส่ง request จริงผ่าน HTTP client
    resp, err := http.Get(ts.URL + "/user?id=999")
    if err != nil {
        t.Fatal(err)
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        t.Errorf("expected 200, got %d", resp.StatusCode)
    }

    body, _ := io.ReadAll(resp.Body)
    if !strings.Contains(string(body), "Integration User") {
        t.Errorf("unexpected body: %s", body)
    }
}

```

☺ สรุปแนวทางปฏิบัติที่ดี

หัวข้อ	คำแนะนำ
Handler Testing	ใช้ <code>httptest.NewRecorder</code> + <code>httptest.NewRequest</code> เสมอ
Dependency Injection	ทำให้ handler รับ interface แทน concrete type เพื่อให้ mock ได้ง่าย
Middleware Testing	สร้าง dummy handler แล้วห่อด้วย middleware ที่ต้องการทดสอบ
Panic Recovery	ใช้ <code>defer</code> ใน middleware และทดสอบด้วย handler ที่ panic
Integration Test	ใช้ <code>httptest.NewServer</code> สร้าง server จริง แต่ไม่ต้อง bind port
Assertions	ตรวจสอบ status code, body, header, และ verify ว่า mock ถูกเรียก

📦 ตัวอย่างโค้ดเพิ่มเติม: การตรวจสอบ Header หรือ Cookie ใน middleware

```
// middleware/auth.go
func AuthMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        token := r.Header.Get("Authorization")
        if token != "secret-token" {
            http.Error(w, "unauthorized", http.StatusUnauthorized)
            return
        }
        next.ServeHTTP(w, r)
    })
}

// ทดสอบ AuthMiddleware
func TestAuthMiddleware_ValidToken(t *testing.T) {
    final := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.WriteHeader(http.StatusOK)
    })
    handler := AuthMiddleware(final)

    req := httptest.NewRequest(http.MethodGet, "/", nil)
    req.Header.Set("Authorization", "secret-token")
    rr := httptest.NewRecorder()

    handler.ServeHTTP(rr, req)
    if rr.Code != http.StatusOK {
        t.Errorf("expected 200, got %d", rr.Code)
    }
}

func TestAuthMiddleware_InvalidToken(t *testing.T) {
    final := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        t.Error("handler should not be called")
    })
    handler := AuthMiddleware(final)
```

```

req := httptest.NewRequest(http.MethodGet, "/", nil)
req.Header.Set("Authorization", "wrong")
rr := httptest.NewRecorder()

handler.ServeHTTP(rr, req)
if rr.Code != http.StatusUnauthorized {
    t.Errorf("expected 401, got %d", rr.Code)
}
}

```

☑ สรุปภาพรวมการทดสอบใน Go Web API

flowchart TD

```

A[เขียน Handler / Middleware] --> B[กำหนด Interface สำหรับ Dependency]
B --> C[สร้าง Mock ของ Dependency]
C --> D[เขียน Unit Test ด้วย httptest]
D --> E[ทดสอบ Handler แต่ละกรณี]
D --> F[ทดสอบ Middleware แต่ละตัว]
D --> G[ทดสอบ Chain Middleware]
G --> H[ทดสอบ Integration ด้วย httptest.Server]
H --> I[รัน go test -cover]

```

ด้วยแนวทางนี้ คุณจะได้อะไรที่ **ทดสอบได้**, **ยืดหยุ่น** และ **บำรุงรักษาง่าย** ตามหลักการ OOP ที่ Go รองรับผ่าน interface และ composition

การทดสอบ WebSocket หรือการทดสอบ graceful shutdown

This chapter dives into two advanced but vital topics for building robust Go applications: testing WebSocket handlers and ensuring your servers shut down gracefully.

1. การทดสอบ WebSocket

การทดสอบ WebSocket นั้นซับซ้อนกว่า HTTP ทั่วไปเพราะต้องจัดการกับการเชื่อมต่อแบบสองทาง (full-duplex) แต่ใน Go เรามีเครื่องมือและเทคนิคที่ช่วยให้ทดสอบได้อย่างมีประสิทธิภาพ

วิธีการทดสอบ WebSocket

มีแนวทางหลักๆ 3 แบบที่ใช้กัน:

1. **Integration Test ด้วย `httptest.Server`** วิธีที่ตรงไปตรงมาและเหมาะกับกรณีส่วนใหญ่ โดยจะสร้าง WebSocket server จริงใน test และทำการเชื่อมต่อไปทดสอบ[reference:0]。
2. **Unit Test ด้วย `Mock Connection`** ออกแบบโค้ดให้พึ่งพา interface (`WSConn`) แทน concrete type (`*websocket.Conn`) เพื่อให้สามารถใช้ mock connection ทดสอบ business logic แบบแยกส่วน[reference:1]。

3. ใช้ **Library** ช่วยในการทดสอบ มี library ที่ออกแบบมาสำหรับการทดสอบ WebSocket โดยเฉพาะ เช่น **wsmock** หรือ **wstest** ซึ่งช่วยลดความซับซ้อนได้มาก[reference:2]。

◇ วิธีที่ 1: Integration Test ด้วย **httptest.Server**

นี่เป็นวิธีพื้นฐานและตรงไปตรงมาที่สุด โดยใช้ **httptest.NewServer** สร้าง test server จากนั้น client ก็ dial ไปหา test server เหมือนเชื่อมต่อจริงๆ เลย[reference:3]。

```
// handler/websocket_handler.go
package handler

import (
    "net/http"
    "github.com/gorilla/websocket"
)

var upgrader = websocket.Upgrader{
    CheckOrigin: func(r *http.Request) bool { return true },
}

func WebSocketHandler(w http.ResponseWriter, r *http.Request) {
    conn, err := upgrader.Upgrade(w, r, nil)
    if err != nil {
        http.Error(w, "Could not upgrade", http.StatusBadRequest)
        return
    }
    defer conn.Close()

    for {
        msgType, msg, err := conn.ReadMessage()
        if err != nil {
            break
        }
        // Echo ข้อความกลับไป
        conn.WriteMessage(msgType, msg)
    }
}
```

```
// handler/websocket_handler_test.go
package handler

import (
    "net/http/httptest"
    "testing"
    "github.com/gorilla/websocket"
)

func TestWebSocketHandler_Echo(t *testing.T) {
    // 1. สร้าง test server
```

```

server := httptest.NewServer(http.HandlerFunc(WebSocketHandler))
defer server.Close()

// 2. แปลง http URL เป็น ws URL
wsURL := "ws" + server.URL[4:] + "/ws"

// 3. Dial ไปยัง test server
conn, _, err := websocket.DefaultDialer.Dial(wsURL, nil)
if err != nil {
    t.Fatalf("Dial error: %v", err)
}
defer conn.Close()

// 4. ส่งข้อความ
testMsg := "Hello, WebSocket!"
err = conn.WriteMessage(websocket.TextMessage, []byte(testMsg))
if err != nil {
    t.Fatalf("Write error: %v", err)
}

// 5. อ่าน response
_, msg, err := conn.ReadMessage()
if err != nil {
    t.Fatalf("Read error: %v", err)
}

if string(msg) != testMsg {
    t.Errorf("Expected %q, got %q", testMsg, msg)
}
}

```

◇ วิธีที่ 2: Unit Test แบบ Mock Connection (Best Practice)

นี่เป็นวิธีที่ถูกหลักการออกแบบซอฟต์แวร์มากที่สุด เพราะช่วยให้เราทดสอบ business logic ได้โดยไม่ต้องพึ่งพา network เลย ขั้นตอนคือกำหนด **interface** สำหรับ connection จากนั้นสร้าง **mock** ที่ implement interface เดียวกันเพื่อใช้ใน unit test[reference:4]。

1. กำหนด WSConn Interface เพื่อใช้ Dependency Injection

```

// websocket/wsconn.go
package websocket

import "time"

// WSConn interface (กำหนดเมธอดที่ business logic ต้องการ)
type WSConn interface {
    ReadMessage() (messageType int, data []byte, err error)
    WriteMessage(messageType int, data []byte) error
    Close() error
}

```

```
    SetReadDeadline(t time.Time) error
}
```

2. สร้าง Wrapper สำหรับ Gorilla's `websocket.Conn` (ใช้ใน production)

```
// websocket/gorilla_wrapper.go
package websocket

import (
    "github.com/gorilla/websocket"
)

type GorillaConn struct {
    conn *websocket.Conn
}

func NewGorillaConn(conn *websocket.Conn) *GorillaConn {
    return &GorillaConn{conn: conn}
}

func (g *GorillaConn) ReadMessage() (int, []byte, error) {
    return g.conn.ReadMessage()
}

func (g *GorillaConn) WriteMessage(messageType int, data []byte) error {
    return g.conn.WriteMessage(messageType, data)
}

func (g *GorillaConn) Close() error {
    return g.conn.Close()
}

func (g *GorillaConn) SetReadDeadline(t time.Time) error {
    return g.conn.SetReadDeadline(t)
}
```

3. สร้าง Mock Connection สำหรับใช้ใน Unit Test

```
// websocket/mock_conn.go
package websocket

import (
    "errors"
    "sync"
    "time"
)
```

```

type MockConn struct {
    mu          sync.Mutex
    sentMessages [][]byte
    messagesToRead [][]byte
    closed      bool
}

func NewMockConn() *MockConn {
    return &MockConn{
        messagesToRead: make([][]byte, 0),
        sentMessages:    make([][]byte, 0),
    }
}

// QueueMessage กำหนด message ที่จะถูกอ่านใน ReadMessage()
func (m *MockConn) QueueMessage(data []byte) {
    m.mu.Lock()
    defer m.mu.Unlock()
    m.messagesToRead = append(m.messagesToRead, data)
}

func (m *MockConn) ReadMessage() (int, []byte, error) {
    m.mu.Lock()
    defer m.mu.Unlock()

    if m.closed {
        return 0, nil, errors.New("connection closed")
    }

    if len(m.messagesToRead) == 0 {
        return 0, nil, errors.New("no messages queued")
    }

    msg := m.messagesToRead[0]
    m.messagesToRead = m.messagesToRead[1:]
    return 1, msg, nil
}

func (m *MockConn) WriteMessage(messageType int, data []byte) error {
    m.mu.Lock()
    defer m.mu.Unlock()

    if m.closed {
        return errors.New("connection closed")
    }

    m.sentMessages = append(m.sentMessages, data)
    return nil
}

func (m *MockConn) Close() error {
    m.mu.Lock()
    defer m.mu.Unlock()

```

```

    m.closed = true
    return nil
}

func (m *MockConn) SetReadDeadline(t time.Time) error {
    return nil // Mock นี้ไม่สน deadline
}

func (m *MockConn) GetSentMessages() [][]byte {
    m.mu.Lock()
    defer m.mu.Unlock()
    return m.sentMessages
}

```

4. Business Logic ที่รับ WSConn Interface

```

// service/chat_service.go
package service

import (
    "encoding/json"
    "log"
    "yourmodule/websocket"
)

type Message struct {
    Username string `json:"username"`
    Content  string `json:"content"`
}

type ChatService struct {
    // ไม่ต้องพึ่งพา concrete type!
}

func (s *ChatService) HandleConnection(conn websocket.WSConn) {
    defer conn.Close()

    for {
        _, data, err := conn.ReadMessage()
        if err != nil {
            log.Printf("Read error: %v", err)
            break
        }

        var msg Message
        if err := json.Unmarshal(data, &msg); err != nil {
            // ส่ง error กลับไปที่ client
            conn.WriteMessage(1, []byte("invalid message format"))
            continue
        }
    }
}

```

```

        // Business logic: broadcast หรือ process ข้อความ
        response, _ := json.Marshal(map[string]string{
            "status": "received",
            "from":   msg.Username,
        })
        conn.WriteMessage(1, response)
    }
}

```

5. Unit Test โดยใช้ Mock Connection

```

// service/chat_service_test.go
package service

import (
    "encoding/json"
    "testing"
    "yourmodule/websocket"
)

func TestChatService_HandleValidMessage(t *testing.T) {
    // 1. สร้าง mock connection
    mockConn := websocket.NewMockConn()

    // 2. จำลอง incoming message
    msg := Message{Username: "Alice", Content: "Hello"}
    data, _ := json.Marshal(msg)
    mockConn.QueueMessage(data)

    // 3. เรียกใช้ service (inject mock)
    svc := &ChatService{}
    svc.HandleConnection(mockConn)

    // 4. ตรวจสอบว่า service ส่ง response อะไรกลับไป
    sent := mockConn.GetSentMessages()
    if len(sent) == 0 {
        t.Fatal("Expected response, got none")
    }

    var response map[string]string
    json.Unmarshal(sent[0], &response)
    if response["status"] != "received" {
        t.Errorf("Expected status 'received', got %v", response["status"])
    }
}

func TestChatService_HandleInvalidMessage(t *testing.T) {
    mockConn := websocket.NewMockConn()
    mockConn.QueueMessage([]byte("invalid json"))
}

```

```

svc := &ChatService{}
svc.HandleConnection(mockConn)

sent := mockConn.GetSentMessages()
if len(sent) == 0 {
    t.Fatal("Expected error response, got none")
}

if string(sent[0]) != "invalid message format" {
    t.Errorf("Expected error message, got %s", sent[0])
}
}

```

◊ วิธีที่ 3: ใช้ Library ช่วยในการทดสอบ

มี library ที่ออกแบบมาเพื่อการทดสอบ WebSocket โดยเฉพาะ ลองดู 2 ตัวนี้:

ตัวที่ 1: wstest - ช่วยให้ทดสอบ WebSocket handler ได้โดยไม่ต้องเปิดพอร์ตจริง[reference:5]。

```

import (
    "testing"
    "github.com/gorilla/websocket"
    "github.com/posener/wstest"
)

func TestWithWSTest(t *testing.T) {
    handler := http.HandlerFunc(WebSocketHandler)
    dialer := wstest.NewDialer(handler)

    conn, _, err := dialer.Dial("ws://whatever/ws", nil)
    if err != nil {
        t.Fatal(err)
    }
    defer conn.Close()

    conn.WriteMessage(websocket.TextMessage, []byte("ping"))
    _, msg, _ := conn.ReadMessage()
    // ตรวจสอบ response...
}

```

ตัวที่ 2: wsmock - ให้ Gorilla mocks สำหรับ unit testing[reference:6]。

```

import (
    "testing"
    "github.com/silently/wsmock"
)

```

```
func TestWithWSMock(t *testing.T) {
    conn, rec := wsmock.NewGorillaMockAndRecorder(t)

    // ส่งข้อความจาก client ไปยัง handler
    conn.Send("Hello")

    // ประกาศ assertion ว่าควรได้รับ response อะไร
    rec.AssertReceived("Hello, World!")

    wsmock.Run(t, 100*time.Millisecond)
}
```

แผนภาพการทดสอบ WebSocket

```
flowchart TB
    subgraph Production["โหมด Production"]
        Client[Client] --> Upgrader[HTTP Handler<br>Upgrade to WebSocket]
        Upgrader --> RealConn["*websocket.Conn"]
        RealConn --> BusinessLogic[Business Logic]
    end

    subgraph Test["Integration Test (httptest)"]
        TestClient[Test Client] --> TestServer["httptest.Server"]
        TestServer --> RealConn2["*websocket.Conn"]
        RealConn2 --> BusinessLogic2[Business Logic]
    end

    subgraph UnitTest["Unit Test (Mock Connection)"]
        MockConn[MockConn<br>implement WSConn interface] --> BusinessLogic3[Business Logic]
        BusinessLogic3 --> Assertions[ตรวจสอบ sentMessages]
    end

    RealConn -.-> WSConnInterface["WSConn Interface"]
    MockConn -.-> WSConnInterface
    BusinessLogic2 -.-> WSConnInterface
    BusinessLogic3 -.-> WSConnInterface
```

2. การทดสอบ Graceful Shutdown

Graceful shutdown คือการที่เซิร์ฟเวอร์หยุดรับ connection ใหม่ แต่ยังคงรอให้ requests ที่กำลังประมวลผลอยู่เสร็จก่อนที่จะปิดตัวเองลง[reference:7]。การทดสอบ graceful shutdown สามารถทำได้หลายระดับ

◇ วิธีที่ 1: Acceptance Test (Build + Run + Send Signal)

นี่เป็นวิธีที่ใช้ใน production จริง วิธีคือสร้าง binary ของเซิร์ฟเวอร์, รันมันขึ้นมา, ส่ง request ที่ใช้เวลานาน, จากนั้นส่งสัญญาณ SIGINT (Ctrl+C) เพื่อดูว่าเซิร์ฟเวอร์ shutdown แบบ graceful หรือไม่[reference:8]。

```

// graceful_test.go
package graceful_test

import (
    "bytes"
    "fmt"
    "net/http"
    "os/exec"
    "testing"
    "time"
)

func TestGracefulShutdown_WithSlowRequest(t *testing.T) {
    // 1. Build binary
    cmd := exec.Command("go", "build", "-o", "testserver", "./cmd/server")
    if err := cmd.Run(); err != nil {
        t.Fatalf("Build failed: %v", err)
    }
    defer exec.Command("rm", "-f", "testserver").Run()

    // 2. Run server in background
    serverCmd := exec.Command("./testserver")
    if err := serverCmd.Start(); err != nil {
        t.Fatalf("Start server failed: %v", err)
    }
    defer serverCmd.Process.Kill()

    // 3. Wait for server to be ready
    time.Sleep(500 * time.Millisecond)

    // 4. Send a slow request (ใน goroutine)
    respChan := make(chan *http.Response, 1)
    errChan := make(chan error, 1)

    go func() {
        // GET request ที่ใช้เวลานาน 10 วินาที
        resp, err := http.Get("http://localhost:8080/slow")
        if err != nil {
            errChan <- err
            return
        }
        respChan <- resp
    }()

    // 5. Give the request some time to start
    time.Sleep(500 * time.Millisecond)

    // 6. Send SIGINT to the server
    if err := serverCmd.Process.Signal(os.Interrupt); err != nil {
        t.Fatalf("Failed to send interrupt: %v", err)
    }
}

```

```
// 7. Wait for result (should succeed if graceful shutdown works)
select {
case resp := <-respChan:
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK {
        t.Errorf("Expected 200, got %d", resp.StatusCode)
    }
    buf := new(bytes.Buffer)
    buf.ReadFrom(resp.Body)
    if buf.String() != "slow request finished" {
        t.Errorf("Unexpected response: %s", buf.String())
    }
case err := <-errChan:
    t.Errorf("Request failed: %v", err)
case <-time.After(15 * time.Second):
    t.Error("Timeout waiting for response")
}
}
```

◇ วิธีที่ 2: Unit Test การทำงานของ Shutdown Logic

เราสามารถทดสอบการทำงานของ `Shutdown()` method ได้โดยตรงด้วย `httptest.Server` ที่รันอยู่ใน test process[reference:9]。

```
// graceful_test.go
package graceful

import (
    "context"
    "net/http"
    "net/http/httptest"
    "testing"
    "time"
)

func TestServer_ShutdownCompletesInflightRequests(t *testing.T) {
    // 1. สร้าง handler ที่ทำงานช้า
    handler := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        time.Sleep(2 * time.Second)
        w.WriteHeader(http.StatusOK)
        w.Write([]byte("done"))
    })

    // 2. สร้าง server พร้อม graceful shutdown wrapper
    srv := &http.Server{
        Addr:    ":0",
        Handler: handler,
    }

    // 3. เริ่ม server ใน goroutine
```

```

errChan := make(chan error, 1)
go func() {
    if err := srv.ListenAndServe(); err != nil && err !=
http.ErrServerClosed {
        errChan <- err
    }
}()
defer srv.Close()

// 4. สร้าง request channel
respChan := make(chan *http.Response, 1)
req, _ := http.NewRequest("GET", "http://localhost"+srv.Addr+"/", nil)

go func() {
    client := &http.Client{}
    resp, err := client.Do(req)
    if err != nil {
        return
    }
    respChan <- resp
}()

// 5. รอให้ request เริ่มทำงาน
time.Sleep(500 * time.Millisecond)

// 6. เรียก Shutdown (graceful shutdown)
ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
defer cancel()

shutdownDone := make(chan struct{})
go func() {
    if err := srv.Shutdown(ctx); err != nil {
        t.Errorf("Shutdown error: %v", err)
    }
    close(shutdownDone)
}()

// 7. รอ response จาก request ที่กำลังทำงานอยู่
select {
case <-shutdownDone:
    t.Error("Shutdown completed too early, before the slow request
finished")
case <-time.After(3 * time.Second):
    // คาดว่า request จะเสร็จภายใน 3 วินาที
}

select {
case resp := <-respChan:
    if resp.StatusCode != http.StatusOK {
        t.Errorf("Expected 200, got %d", resp.StatusCode)
    }
case <-time.After(2 * time.Second):
    t.Error("Request did not complete after shutdown")
}

```

```

}

// 8. รอให้ shutdown เสร็จ
<-shutdownDone
}

```

◇ วิธีที่ 3: ทดสอบ Shutdown Context Cancellation (Signal.NotifyContext)

ใช้ `signal.NotifyContext` เพื่อทดสอบการตอบสนองต่อสัญญาณ `interrupt[reference:10]`。

```

// main_test.go
package main

import (
    "context"
    "os"
    "os/signal"
    "syscall"
    "testing"
    "time"
)

func TestMain_ShutdownOnSignal(t *testing.T) {
    // สร้าง context ที่จะถูก cancel เมื่อได้รับ signal
    ctx, stop := signal.NotifyContext(context.Background(), syscall.SIGINT,
    syscall.SIGTERM)
    defer stop()

    // จำลองการส่ง signal ไปยัง context
    done := make(chan struct{})
    go func() {
        <-ctx.Done()
        close(done)
    }()

    // ส่ง signal เอง
    p, err := os.FindProcess(os.Getpid())
    if err != nil {
        t.Fatalf("Failed to find process: %v", err)
    }
    if err := p.Signal(syscall.SIGINT); err != nil {
        t.Fatalf("Failed to send signal: %v", err)
    }

    // รอให้ context ถูก cancel
    select {
    case <-done:
        // success
    case <-time.After(1 * time.Second):
        t.Error("Context was not canceled after signal")
    }
}

```

```
}  
}
```

แผนภาพการทำงานของ Graceful Shutdown

```
sequenceDiagram  
    participant U as User  
    participant K as Kubernetes/System  
    participant S as Go HTTP Server  
    participant H as HTTP Handler  
  
    U->>S: Request 1 (slow, 5 sec)  
    activate S  
    S->>H: Process Request 1  
    activate H  
  
    K->>S: SIGTERM  
    S->>S: Stop accepting new connections  
  
    U->>S: Request 2 (new)  
    S-->>U: Connection refused  
  
    Note over S,H: Request 1 still processing  
    H-->>S: Response completed  
    deactivate H  
    S-->>U: Response sent  
  
    S->>S: Shutdown complete  
    deactivate S
```

☒ สรุปแนวทางปฏิบัติที่ดี

หัวข้อ	คำแนะนำ
WebSocket Testing	ใช้ interface (WSConn) เพื่อ dependency injection จะทำให้ทดสอบ business logic ได้ง่ายที่สุด
Integration Test	ใช้ httptest.NewServer + Gorilla WebSocket client สำหรับ end-to-end testing
Mock Library	พิจารณาใช้ wstest (ไม่ต้องเปิดพอร์ต) หรือ wsmock (มี assertion helper)
Graceful Shutdown	ใช้ http.Server.Shutdown() ร่วมกับ signal.NotifyContext ใน production
Acceptance Test	สร้าง binary → รัน → ส่ง request เข้า → ส่ง SIGTERM → รอ response
Unit Test Shutdown	เรียก Shutdown() โดยตรงใน test และตรวจสอบว่า in-flight requests เสร็จสมบูรณ์

- ด้วยเทคนิคเหล่านี้ คุณสามารถมั่นใจได้ว่า WebSocket service ของคุณทำงานได้อย่างถูกต้องและสามารถ shutdown แบบไม่กระทบต่อผู้ใช้งานจริงใน production ได้

5. Struct

Struct คืออะไร?

Struct (structure) เป็นชนิดข้อมูล (type) ที่กำหนดขึ้นเอง โดยรวมฟิลด์ (fields) หลายชนิดเข้าเป็นหน่วยเดียวกัน ใช้แทน "record" หรือ "object" ในภาษา Go

Struct มีกี่แบบ?

- **Named struct** – ประกาศด้วย `type Name struct { fields }`
- **Anonymous struct** – ประกาศตรงจุดโดยไม่ตั้งชื่อ
- **Embedded struct** – ผัง struct อื่นโดยไม่ระบุชื่อฟิลด์ (ใช้แทน inheritance)
- **Empty struct** – `struct{}` ไม่มีฟิลด์ ใช้เป็นเซตหรือ signalling

ใช้อย่างไร ในกรณีไหน?

- จัดกลุ่มข้อมูลที่มีโครงสร้าง (User, Product, Order)
- จัดกลุ่มข้อมูลที่เกี่ยวข้องกัน
- ผัง struct เพื่อ reuse โค้ด (composition)
- สร้าง method ให้ struct (รับ receiver)

หลักการทำงาน

1. ประกาศ struct type พร้อมฟิลด์พร้อมชนิด
2. สร้าง instance ด้วย `var, new, &StructName{...}`
3. เข้าถึงฟิลด์ด้วย dot (`.`)
4. สามารถเพิ่ม method ให้ struct ด้วย receiver (value หรือ pointer)

Dataflow Diagram (Flowchart TB) - Struct with Methods

```
graph TB
    A[ประกาศ type User struct] --> B[สร้าง instance u]
    B --> C[เรียก u.GetFullName]
    C --> D[รับค่า firstName + lastName]
```

ตัวอย่างการใช้งานจริง: ระบบจัดการผู้ใช้

```
package main

import "fmt"
```

```

type User struct {
    ID          int
    FirstName   string
    LastName    string
    Email       string
}

// Method with value receiver
func (u User) FullName() string {
    return u.FirstName + " " + u.LastName
}

// Method with pointer receiver (modify)
func (u *User) UpdateEmail(newEmail string) {
    u.Email = newEmail
}

func main() {
    // create struct
    user := User{
        ID:          1,
        FirstName:   "John",
        LastName:    "Doe",
        Email:       "john@example.com",
    }

    fmt.Println(user.FullName()) // John Doe

    user.UpdateEmail("john.doe@example.com")
    fmt.Println(user.Email) // john.doe@example.com
}

```

เทมเพลต: Embedded struct (Composition)

```

type Base struct {
    CreatedAt time.Time
    UpdatedAt time.Time
}

type Product struct {
    Base // embedded
    ID   int
    Name string
    Price float64
}

```

6. Inheritance

Inheritance คืออะไร?

Inheritance (การสืบทอด) เป็นกลไกใน OOP ที่คลาสลูกสามารถสืบทอดฟิลด์และเมธอดจากคลาสแม่ ทำให้สามารถ reuse โค้ดและสร้างความสัมพันธ์แบบ **is-a** Go **ไม่มี** inheritance แบบคลาส แต่ใช้ **embedding** (struct ผึง struct) และ **interface** เพื่อให้ได้ผลคล้ายกันโดยเน้น **composition** (has-a) มากกว่า

Inheritance มีกี่แบบ?

ในภาษา OOP ทั่วไป:

- **Single inheritance** – สืบทอดจากคลาสแม่เดียว (Java, C#)
- **Multiple inheritance** – สืบทอดจากหลายคลาส (C++) – เกิดปัญหา diamond problem
- **Multilevel inheritance** – A -> B -> C

Go ใช้ **embedding** ซึ่งไม่ใช่ inheritance แท้ แต่ให้ความสามารถในการ reuse โค้ดและ method promotion

ใช้อย่างไร ในกรณีไหน?

- ต้องการ reuse โค้ดโดยไม่ต้องเขียนซ้ำ (ใช้ embedding)
- ต้องการสร้าง "type hierarchy" แบบง่ายผ่าน interface
- หลีกเลี่ยงปัญหาความซับซ้อนของ deep inheritance
- ใช้ในไลบรารีเช่น GORM (ผึง `gorm.Model`)

หลักการทำงาน (ใน Go ด้วย embedding)

1. ประกาศ struct `Parent` ที่มีฟิลด์และเมธอด
2. ประกาศ struct `Child` ที่ผึง `Parent` (anonymous field)
3. Child สามารถเข้าถึงฟิลด์และเมธอดของ Parent โดยตรง (promotion)
4. Child สามารถ override เมธอดได้โดยการประกาศเมธอดชื่อเดียวกัน
5. ไม่สามารถแปลงจาก Child เป็น Parent ได้โดยอัตโนมัติ (ต้องใช้ interface)

Dataflow Diagram (Flowchart TB) - Embedding

```
graph TB
    A["type Animal struct"] --> B["type Dog struct<br/>Animal"]
    B --> C["dog := Dog{...}"]
    C --> D["เรียก dog.Speak()<br/>พบเมธอดใน Dog?"]
    D -- มี --> E["เรียก Dog.Speak"]
    D -- ไม่มี --> F["เรียก Animal.Speak"]
```

ตัวอย่างการใช้งานจริง: ระบบพาหนะ

```
package main

import "fmt"

// Base struct
```

```

type Vehicle struct {
    Brand string
    Year  int
}

func (v Vehicle) Info() string {
    return fmt.Sprintf("%s (%d)", v.Brand, v.Year)
}

func (v Vehicle) Start() string {
    return "Engine started"
}

// Car embeds Vehicle
type Car struct {
    Vehicle
    Doors int
}

// Override Start method
func (c Car) Start() string {
    return "Car engine started with key"
}

// Motorcycle embeds Vehicle
type Motorcycle struct {
    Vehicle
    HasSidecar bool
}

func main() {
    car := Car{
        Vehicle: Vehicle{Brand: "Toyota", Year: 2020},
        Doors: 4,
    }

    bike := Motorcycle{
        Vehicle: Vehicle{Brand: "Harley", Year: 2019},
        HasSidecar: false,
    }

    fmt.Println(car.Info()) // Toyota (2020) (promoted)
    fmt.Println(car.Start()) // Car engine started with key (overridden)
    fmt.Println(bike.Start()) // Engine started (inherited)
}

```

เทมเพลต: Embedding แบบ GORM

```
import "gorm.io/gorm"
```

```

type BaseModel struct {
    ID          uint          `gorm:"primaryKey"`
    CreatedAt   time.Time
    UpdatedAt   time.Time
    DeletedAt   gorm.DeletedAt `gorm:"index"`
}

type User struct {
    BaseModel // embedding
    Name      string
    Email     string
}
// User จะมีฟิลด์ ID, CreatedAt, UpdatedAt, DeletedAt โดยอัตโนมัติ

```

สรุปเปรียบเทียบ

กระบวนทัศน์	จุดเน้น	Go รองรับอย่างไร
Procedural	ลำดับขั้นตอน, ฟังก์ชัน	พื้นฐานของ Go (main, functions, packages)
Concurrent	การทำงานหลายอย่างพร้อมกัน	goroutine, channel, sync
Functional	Pure functions, immutability, higher-order	first-class functions, closures, generics
OOP	วัตถุ, encapsulation, polymorphism	struct (data) + interface (behavior), composition
Struct	การรวมข้อมูล	type struct, methods, embedding
Inheritance	สืบทอดคุณสมบัติ (is-a)	ไม่มี direct inheritance แต่ใช้ embedding และ interface

แต่ละกระบวนทัศน์มีข้อดีและเหมาะกับงานต่างกัน Go ออกแบบให้เรียบง่ายและสามารถผสมผสานหลายกระบวนทัศน์ได้อย่างลงตัว โดยยึดหลัก "less is more" และ "composition over inheritance"

แหล่งอ้างอิง

- [The Go Programming Language Specification](#)
- [Effective Go](#)
- [Go by Example](#)
- [GORM Documentation](#)

1.14 ขั้นตอนการพัฒนาโปรแกรม

1. เขียนซอร์สโค้ด (.go)
2. คอมไพล์ (go build)
3. ทดสอบรัน (go run)

4. แก้ไขข้อผิดพลาด (debug)
5. จัดการแพ็คเกจ (go mod)
6. ทดสอบหน่วย (go test)

Heap, Worker, Pool คืออะไร?

1. Heap (ฮีป)

Heap มี 2 ความหมายในบริบทการเขียนโปรแกรม:

1. โครงสร้างข้อมูล **Heap** (Priority Queue)

- เป็นต้นไม้ไบนารีที่สมบูรณ์ (complete binary tree) ซึ่งมีคุณสมบัติ **heap property**
 - **Max-heap**: โหนดพ่อมีค่ามากกว่าหรือเท่ากับลูก
 - **Min-heap**: โหนดพ่อมีค่าน้อยกว่าหรือเท่ากับลูก
- ใช้สำหรับสร้าง **priority queue** (คิวลำดับความสำคัญ)
- การดำเนินการหลัก: **Push** (แทรก) และ **Pop** (ดึงค่าสูงสุด/ต่ำสุด) ด้วยเวลา $O(\log n)$

2. **Heap Memory** (หน่วยความจำฮีป)

- พื้นที่หน่วยความจำที่ใช้สำหรับ allocate วัตถุที่มีอายุการใช้งานยาวนาน (เช่น ตัวแปรที่สร้างด้วย **new**, **make** ใน Go)
- ต่างจาก stack memory ซึ่งเป็นหน่วยความจำเฉพาะฟังก์ชัน

ในบริบทของ Queue Processor ที่พูดถึงก่อนหน้านี้ **heap** หมายถึง priority queue ที่ใช้จัดลำดับงานตามความสำคัญ

2. Worker (เวิร์กเกอร์)

Worker คือหน่วยงาน (goroutine, thread, หรือ process) ที่ทำหน้าที่ **ดึงงานจากคิวและประมวลผล**

- โดยทั่วไป Worker จะทำงานแบบไม่มีที่สิ้นสุด (loop) รอรับงาน
- ช่วยให้ระบบสามารถทำงานหลายอย่างพร้อมกัน (concurrency)
- การมีหลาย Workers เรียกว่า **Worker Pool**

3. Pool (พูล)

Pool คือกลุ่มของทรัพยากรที่เตรียมไว้ล่วงหน้าเพื่อนำมาใช้ซ้ำ ลดค่าใช้จ่ายในการสร้างและทำลายทรัพยากรบ่อย ๆ

- **Worker Pool**: กลุ่มของ Workers ที่พร้อมรับงาน
- **Connection Pool**: กลุ่มของ connection ฐานข้อมูล
- **Memory Pool**: จัดสรรบล็อกหน่วยความจำล่วงหน้า

แผนภาพประกอบ (Flowchart + Worker Pool + Priority Queue)

```
graph TB
    subgraph Producer
```

```

    A[Client / API] --> B[Create Job with Priority]
end

subgraph PriorityQueue[Heap-based Priority Queue]
    C[(Min-Heap / Max-Heap)]
    B --> C
end

subgraph WorkerPool[Worker Pool]
    D[Worker 1] --> E[Process Job]
    F[Worker 2] --> E
    G[Worker 3] --> E
    H[...]
end

C -- Job with highest priority --> D
C -- Job with highest priority --> F
C -- Job with highest priority --> G

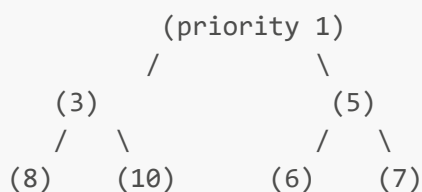
E --> I[Database / External Service]
I --> J[Result / Acknowledge]

```

คำอธิบาย:

- **Producer** สร้างงานและกำหนด priority
- งานจะถูกแทรกเข้าไปใน **Priority Queue** (implemented ด้วย heap) ซึ่งจะจัดเรียงตาม priority
- **Worker Pool** มี Workers หลายตัว พร้อมดึงงานจากคิว (pop ค่าที่ priority สูงสุด)
- แต่ละ Worker ประมวลผลงานและส่งผลลัพธ์กลับ

ตัวอย่างการทำงานของ Heap ใน Priority Queue (Min-Heap)



- เมื่อ **Push** งาน priority 2 → แทรกที่ตำแหน่งท้าย แล้วปรับโครงสร้าง (heapify up)
- เมื่อ **Pop** งาน priority ต่ำสุด → ดึงราก (priority 1) แล้วนำโหนดสุดท้ายมาเป็นรากใหม่ แล้วปรับโครงสร้าง (heapify down)

ตัวอย่าง Worker Pool ใน Go

```

type Job struct {
    ID      int

```

```

    Priority int
    Payload interface{}
}

func worker(jobs <-chan Job, wg *sync.WaitGroup) {
    defer wg.Done()
    for job := range jobs {
        fmt.Printf("Worker processing job %d (priority %d)\n", job.ID,
job.Priority)
        // simulate work
        time.Sleep(100 * time.Millisecond)
    }
}

func main() {
    const numWorkers = 3
    jobs := make(chan Job, 100)

    var wg sync.WaitGroup
    for i := 0; i < numWorkers; i++ {
        wg.Add(1)
        go worker(jobs, &wg)
    }

    // ส่งงานเข้า queue (ในตัวอย่างใช้ channel ธรรมดา แต่ priority queue ต้อง implement
เอง)
    for i := 0; i < 10; i++ {
        jobs <- Job{ID: i, Priority: i % 3}
    }
    close(jobs)
    wg.Wait()
}

```

สรุป

คำศัพท์	ความหมาย	การประยุกต์ใน Queue Processor
Heap	โครงสร้างข้อมูลแบบ priority queue	ใช้จัดเรียงงานตามลำดับความสำคัญก่อนดึงไปประมวลผล
Worker	หน่วยประมวลผล (goroutine/thread)	ดึงงานจากคิวและทำงานจริง
Pool	กลุ่มทรัพยากรที่เตรียมไว้ล่วงหน้า	Worker Pool ช่วยควบคุม concurrency, reuse goroutines

การรวมกันของ **Heap + Worker Pool** ช่วยให้ระบบสามารถจัดการงานที่มี priority ต่างกันได้อย่างมีประสิทธิภาพ โดยไม่ต้องสร้าง goroutine ใหม่ทุกครั้ง และประมวลผลงานสำคัญก่อนเสมอ

บทที่ 2: รู้จักกับภาษา Go

2.1 ประวัติความเป็นมา

ภาษา Go (หรือ Golang) ถูกพัฒนาโดย Google เริ่มต้นในปี 2007 โดย Robert Griesemer, Rob Pike, และ Ken Thompson เปิดตัวเป็นโอเพนซอร์สในปี 2009 จุดประสงค์เพื่อแก้ปัญหาที่เกิดขึ้นในภาษา C++ และ Java ในระบบขนาดใหญ่ของ Google เช่น การคอมไพล์ที่ช้า, การจัดการการทำงานพร้อมกันที่ซับซ้อน, และความยุ่งยากในการบำรุงรักษา

2.2 จุดเด่นของภาษา Go

- **เรียบง่ายและอ่านง่าย** : ไวยากรณ์กระชับ ไม่มีฟีเจอร์ที่ซับซ้อนเกินจำเป็น
- **คอมไพล์เร็ว** : สามารถคอมไพล์โปรเจกต์ขนาดใหญ่ได้ในไม่กี่วินาที
- **การจัดการหน่วยความจำอัตโนมัติ** : มี garbage collector ที่มีประสิทธิภาพ
- **Concurrency ระดับภาษา** : goroutine และ channel ทำให้เขียนโปรแกรม concurrent ได้ง่าย
- **Static typing** : ตรวจสอบชนิดข้อมูลตั้งแต่ตอนคอมไพล์ ป้องกันข้อผิดพลาด
- **เครื่องมือที่ครบครัน** : go fmt, go test, go mod, go vet, go doc
- **สามารถคอมไพล์ข้ามแพลตฟอร์ม** (cross-compile) ไปยัง Windows, Linux, macOS, ARM เป็นต้น

2.3 โครงสร้างภาษา Go

- ไม่มี class แต่ใช้ struct และ method
- ไม่มี inheritance แต่ใช้ composition และ interface
- ไม่มี exception handling แต่ใช้ error return value
- มี garbage collection
- มี pointer แต่ไม่มี pointer arithmetic

2.4 ตัวอย่าง Hello World ใน Go

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, World!")
}
```

ข้อดีของ Golang

1. การทำงานพร้อมกัน (Concurrency) ที่ยอดเยี่ยม

มี goroutine ซึ่งเบากว่า thread ทั่วไปมาก และ channel สำหรับสื่อสารระหว่าง goroutine ทำให้เขียนโปรแกรม concurrent ได้ง่ายและมีประสิทธิภาพ

2. คอมไพล์เร็วมาก

โค้ด Go คอมไพล์เป็น binary ไฟล์เดียว ใช้เวลาไม่กี่วินาที แม้โปรเจกต์ใหญ่ ช่วยให้ development cycle รวดเร็ว

3. ไวยากรณ์เรียบง่าย อ่านง่าย

ไม่มีฟีเจอร์ซับซ้อนอย่าง inheritance, generics (แต่ปัจจุบันมีแล้วแบบจำกัด), ternary operator ทำให้โค้ดตรงไปตรงมา เรียนรู้ได้ไว

4. ประสิทธิภาพดี

เป็น compiled language + garbage collection ที่ปรับแต่งมาดี ทำงานเร็วใกล้เคียง C/C++ ในหลายงาน

5. Standard library ครบครัน

โดยเฉพาะด้าน networking, HTTP, encoding, crypto ช่วยให้เขียน backend, web server, API ได้โดยไม่ต้องพึ่งพาไลบรารีภายนอกมาก

6. เครื่องมือพัฒนาคุณภาพสูง

มี `go fmt` จัดรูปแบบโค้ดอัตโนมัติ, `go test` สำหรับเทสต์, `go mod` สำหรับจัดการ dependency, `go vet` ตรวจสอบข้อผิดพลาด

7. Cross-platform compile ง่าย

ตั้งค่า `GOOS` และ `GOARCH` ก็คอมไพล์ข้ามระบบปฏิบัติการและสถาปัตยกรรมได้ทันที

ข้อเสียของ Golang

1. Error handling ซ้ำซาก

ต้องตรวจสอบ error ทุกครั้งผ่าน multiple return values ทำให้โค้ดยาวและมี `if err != nil` เยอะ ขาดกลไก exception หรือ structured error handling ที่กระชับ

2. Generics มาช้าและยังมีข้อจำกัด

แม้เวอร์ชัน 1.18+ จะเพิ่ม generics แล้ว แต่ก็ซับซ้อนกว่าและรองรับน้อยกว่าภาษาอื่น (ไม่มี method generics, type constraints ยังไม่สมบูรณ์)

3. ขาดการสืบทอด (inheritance)

ใช้ composition แทน ซึ่งอาจไม่ถนัดสำหรับนักพัฒนาที่คุ้นเคย OOP อย่าง Java หรือ C++ และต้องปรับ mindset

4. Package management ในอดีตยุ่งยาก

ปัจจุบัน go modules ใช้ได้ดีแล้ว แต่ก็ยังมีปัญหาเรื่อง dependency และ versioning บางกรณี โดยเฉพาะถ้ามี replace หรือ indirect dependencies

5. Garbage collection อาจกระทบ real-time

แม้ GC จะเร็วขึ้นมาก แต่ก็ยังมี stop-the-world ช่วงสั้นๆ ไม่เหมาะกับระบบที่ต้องการ latency ต่ำมากๆ เช่น trading, เกม

6. ระบบ GUI และมือถือยังอ่อน

ไม่มี native binding สำหรับ GUI บน desktop หรือ mobile ที่แข็งแรง ไลบรารีก็ยังไม่ค่อยมี Go สำหรับ backend หรือ CLI เป็นหลัก

7. ขาดฟีเจอร์ syntactic sugar บางอย่าง

เช่น ternary operator, while loop (ใช้ for แทน), function overloading ทำให้บางคนรู้สึกว่าจะต้องเขียนโค้ดยืดเยื้อกว่าภาษาอื่น

8. ชุมชนและ ecosystem น้อยกว่า Java, Python, JS

แม้จะเติบโตเร็ว แต่ไลบรารีในบาง domain (data science, ML, GUI, game) ยังมีให้เลือกน้อย

สรุป: Go เหมาะสำหรับงานระบบ backend, microservice, CLI tool, network programming ที่ต้องการประสิทธิภาพสูงและ concurrent ดี แต่ถ้าต้องการ OOP เต็มรูปแบบ, GUI, หรือ rapid prototyping ที่ error handling กระชับ อาจต้องพิจารณาภาษาอื่นประกอบ

ภาษา Go (Golang) เป็นภาษาเอนกประสงค์ที่ถูกออกแบบมาให้เรียบง่าย ทำงานได้เร็ว และจัดการงานพร้อมกัน (Concurrency) ได้ดีเยี่ยม ทำให้ปัจจุบัน Go ถูกนำไปใช้พัฒนาซอฟต์แวร์หลากหลายประเภท ตั้งแต่ระบบระดับองค์กรไปจนถึงเครื่องมือ CLI เล็ก ๆ

นี่คือสิ่งที่คุณสามารถทำได้กับ Go ในโลกจริง:

1. พัฒนา Web Backend / REST API / Microservices

Go คือตัวเลือกอันดับต้น ๆ สำหรับการสร้าง API และบริการบนเซิร์ฟเวอร์ ด้วยความเร็วสูงและการใช้หน่วยความจำน้อย

- **Web Framework ยอดนิยม:** Gin, Echo, Fiber (เร็วระดับใกล้เคียง Node.js/Express แต่ใช้ทรัพยากรน้อยกว่า)
- **Microservices:** Go มีน้ำหนักเบา, compile เป็นไฟล์ไบนารีเดียว ทำให้ deploy ด้วย Docker ได้ง่าย เหมาะกับสถาปัตยกรรมแบบ Distributed Systems
- **GraphQL:** มี library เช่น [gqlgen](#) สำหรับสร้าง GraphQL server

2. เครื่องมือ Command Line (CLI) และ DevOps

Go ถูกใช้สร้างเครื่องมือ CLI ระดับโลกมากมาย เพราะ compile เป็นไฟล์执行ไฟล์เดียว ไม่ต้องพึ่งพา runtime อะไรเพิ่ม

- ตัวอย่างเครื่องมือที่เขียนด้วย **Go:** Docker, Kubernetes, Terraform, Prometheus, Hugo
- ไลบรารียอดนิยม: [cobra](#) (สร้าง CLI แบบ professional), [urfave/cli](#)

3. System Programming และ Network Software

Go มี standard library ที่แข็งแกร่งเรื่อง net/http, tcp/udp, และ low-level I/O ทำให้เหมาะกับ:

- **Proxy, Load Balancer, API Gateway** (เช่น Traefik, Caddy)
- เซิร์ฟเวอร์ **TCP/UDP** ความเร็วสูง
- **WebSocket server**
- **DNS server, VPN tool**

4. การประมวลผลแบบ Concurrent และ Parallel

Go มี goroutine (lightweight thread) และ channel ทำให้เขียนโปรแกรมที่ทำงานพร้อมกันหลายล้านงานได้ง่ายและปลอดภัยกว่า threading แบบดั้งเดิม

- **Real-time data processing**
- **Web crawler / scraper**
- ระบบ **Chat, Gaming server**
- **ETL pipeline**

5. Cloud Native, Distributed Systems

Go คือภาษาหลักของระบบคลาวด์ยุคใหม่

- พัฒนา **Operator** สำหรับ **Kubernetes** (ใช้ client-go)
- **Serverless function** (AWS Lambda, GCP Cloud Functions รองรับ Go)
- **Service Mesh** (เช่น Istio เขียนด้วย Go)

6. เขียน Desktop Application (น้อย แต่ทำได้)

แม้จะไม่ใช่เป้าหมายหลัก แต่ก็มีส่วนช่วย:

- **Fyne** หรือ **Gio** สำหรับ GUI ข้ามแพลตฟอร์ม
- **Wails** – ใช้ Go เป็น backend และ HTML/JS เป็น frontend (คล้าย Electron แต่เบากว่า)

7. เกมและกราฟิก (เฉพาะส่วน server หรือ game logic)

- เกมแนวเรียลไทม์ที่ต้องการความเร็วสูงและการผู้เล่นพร้อมกันจำนวนมาก (Game server)
- มี library เช่น **ebiten** สำหรับทำเกม 2D ได้พอสมควร

8. Blockchain และ Cryptocurrency

- **Ethereum (Geth)** – โคลเอนต์หลักของ Ethereum เขียนด้วย Go
- **Hyperledger Fabric** – blockchain สำหรับองค์กร
- กระเป๋าเงิน, เครื่องมือเกี่ยวกับ crypto

9. ประมวลผลข้อมูล, Machine Learning (ระดับต้น)

- Go ไม่โดดเด่นเท่า Python แต่ก็มี ecosystem พอสมควร: **gonum** (matrix, statistics), **gorgonia** (deep learning คล้าย TensorFlow)
- เหมาะกับการนำโมเดลที่เทรนแล้วไปใช้งานเป็น service (inference)

10. เขียน C/C++ extension และระบบฝังตัว (Embedded)

- ผ่าน **cgo** เรียกใช้ไลบรารี C ได้
- รันบน Raspberry Pi, ARM, หรือ IoT device ขนาดเล็ก (แม้จะกิน resource มากกว่า Rust/C เล็กน้อย)

จุดเด่นที่ทำให้ Go ทำสิ่งเหล่านี้ได้ดี

- **ง่าย เรียนรู้เร็ว:** syntax คล้าย C แต่ปลอดภัยกว่า มี garbage collection
- **compile เร็วมาก:** แม้โปรเจกต์ใหญ่ก็ไม่กี่วินาที
- **ออกแบบมาสำหรับ multicore:** concurrency เป็นส่วนหนึ่งของภาษา
- **Standard library มหาศาล:** มี net/http, json, crypto, template, testing ในตัว
- **Deploy ง่ายสุด ๆ:** compile เป็นไฟล์เดียว (static binary) ลงบนเซิร์ฟเวอร์ได้เลย

สรุป: Go เหมาะกับใคร?

- นักพัฒนาระบบคลาวด์, DevOps, SRE
- คนที่เขียน backend API ต้องการ performance สูง
- ทีมที่ต้องการภาษาเรียนง่าย แต่แข็งแกร่งระดับ production
- โปรเจกต์ที่ต้องขยาย scale แนวนอน (horizontal scaling) หลายล้าน concurrent connection

ถ้าคุณกำลังมองหาภาษาที่ **ทำงานเร็วเท่า C#/Java** แต่ **เขียนง่ายเท่า Python** และ **จัดการ concurrency ได้เป็นระบบ** Go คือตัวเลือกที่ไม่ควรพลาด

Go (Golang) กับงาน IoT – เป็นคู่ที่ลงตัวมากในหลายระดับ โดยเฉพาะในส่วนของ **Edge Gateway**, **เซิร์ฟเวอร์กลาง (Cloud/Backend)** และ **อุปกรณ์ที่มีทรัพยากรพอสมควร** (เช่น Raspberry Pi, BeagleBone, อุปกรณ์ Linux ARM)

1. Go ใช้ทำอะไรใน IoT บ้าง?

ระดับ	ตัวอย่างงาน	ความเหมาะสมของ Go
Edge Gateway / Hub	รวมข้อมูลจากเซ็นเซอร์หลายตัว, ประมวลผลเบื้องต้น, ส่งขึ้นคลาวด์	★★★★★ ยอดเยี่ยม
Backend / Cloud	รับข้อมูลจากอุปกรณ์นับล้าน, API, Dashboard, MQTT broker	★★★★★ ยอดเยี่ยม
อุปกรณ์ปลายทาง (Device)	เซ็นเซอร์ธรรมดา, แอคทูเอเตอร์, ไมโครคอนโทรลเลอร์	★★ พอใช้ (ถ้าใช้ TinyGo)
Real-time control	ควบคุมหุ่นยนต์, ตอบสนองระดับมิลลิวินาที	★★ (มี GC อาจไม่เหมาะกับ hard real-time)

2. จุดเด่นของ Go สำหรับ IoT

☑ Cross-compile ง่ายมาก

`GOOS=linux GOARCH=arm GOARM=7 go build`

ได้ binary ไฟล์เดียว วางรันบน Raspberry Pi, Orange Pi, หรือ router ที่ใช้ MIPS ได้ทันที

☑ Concurrency ระดับสูง

ใช้ goroutine จัดการเซ็นเซอร์หลายตัว, MQTT topic หลายช่อง, HTTP request พร้อมกัน โดยไม่ต้องมานั่งเขียน threading ให้ปวดหัว

☑ ประสิทธิภาพดี หน่วยความจำน้อย

binary เล็ก (ประมาณ 5-20 MB) ใช้ RAM น้อยกว่า Java หรือ Python มาก เหมาะกับ gateway ที่มี RAM 256-512 MB

☑ Standard library ครบคลุม

มี net/http, mqtt (มี library ชั้่นนำ), serial port, json, websocket ในตัว

☑ เสถียรภาพระดับ production

Docker, Kubernetes, Prometheus เขียนด้วย Go – พิสูจน์แล้วว่ารับภาระหนักได้

3. ข้อควรระวัง / ข้อจำกัด

⚠ Garbage Collection (GC)

แม้ Go จะ GC เร็วมาก แต่ก็ยังเกิด latency spike เป็นบางช่วง (ปกติ 10-100 μ s ถึง ms) ถ้าต้องการควบคุมหุ่นยนต์หรือเซ็นเซอร์ที่ต้องการความแม่นยำระดับไมโครวินาที ควรใช้ C หรือ Rust

⚠ ขนาด binary

เทียบกับ C แล้ว Go ใหญ่กว่า (~1-2 MB ขั้นต่ำ) อาจไม่เหมาะกับไมโครคอนโทรลเลอร์หน่วยความจำ 32KB

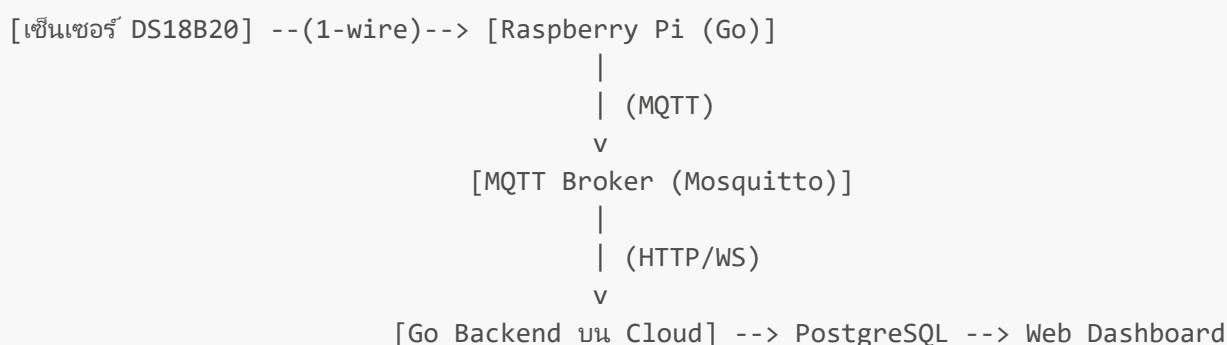
⚠ Driver รองรับฮาร์ดแวร์น้อยกว่า Python/C

การติดต่อ GPIO, I2C, SPI, ADC ใน Go มี library แต่ไม่หนาเท่า `RPi.GPIO` ของ Python หรือ `libc` ของ C

4. เครื่องมือและไลบรารีเด่น

ชื่อ	ใช้ทำอะไร
TinyGo	คอมไพล์ Go ไปรันบน Arduino, ESP32, STM32, RP2040 (ใช้ LLVM แทน GC ปกติ)
Gobot	เฟรมเวิร์ก IoT คล้าย Johnny-Five รองรับ drone, robot, เซ็นเซอร์หลายร้อยตัว
Periph.io	ขับ GPIO, I2C, SPI บน Linux embedded (Raspberry Pi, Jetson Nano, etc.)
Eclipse Paho MQTT	MQTT client สำหรับ Go
Serial	go.bug.st/serial สำหรับอ่านค่าเซ็นเซอร์ผ่าน USB/COM port

5. ตัวอย่างโครงสร้างระบบ IoT ด้วย Go



หรือสำหรับอุปกรณ์ขนาดเล็ก:

Arduino + TinyGo → อ่านค่า → ส่ง serial → Edge Gateway (Go) → ประมวลผล → ส่งขึ้นคลาวด์

6. ภาษาไหนดีที่สุดสำหรับ IoT? (เทียบคร่าวๆ)

ภาษา	เหมาะกับ
C	Firmware, microcontroller หน่วยความจำต่ำ, hard real-time
C++	Arduino, ESP32, Robotics
Python	Prototyping ด่วน, Data analysis, บน Raspberry Pi ที่มีพลัง
Go	Gateway, Backend, ระบบที่ต้องการความเสถียรและ concurrency สูง
Rust	ปลอดภัย, ไร้ GC, กำลังมาแรงใน embedded สมัยใหม่

สรุป: ใช้ Go ใน IoT เมื่อใด?

☒ การใช้ Go ถ้า:

- คุณทำ Edge Gateway, MQTT aggregator, API proxy
- ต้องการ performance สูง แต่ไม่อยากเขียน C
- มีอุปกรณ์ Linux ARM (Raspberry Pi ขึ้นไป)
- ต้องการ deploy เป็น container หรือ binary ง่ายๆ

✗ ไม่เหมาะถ้า:

- ใช้ MCU ราคาถูก (ATmega, ESP8266 แรม 80KB)
- ต้องการ hard real-time ระดับ $<10\ \mu\text{s}$
- มี driver เฉพาะที่เขียนใน Python หรือ C อยู่แล้ว และไม่มี wrapper Go

การจะเริ่มต้นโปรเจกต์ IoT ด้วย Go ให้สำเร็จได้นั้น หัวใจสำคัญคือการเลือกเส้นทางให้ตรงกับโปรเจกต์ เพราะ Go มีความสามารถสองแบบหลัก ๆ คือ การพัฒนา **"ระบบปฏิบัติการ" (Edge Gateway/Backend)** ด้วย Go เต็มรูปแบบ และการพัฒนา **"ตัวอุปกรณ์" (Microcontroller)** โดยตรงด้วย TinyGo

1. เลือกฮาร์ดแวร์ให้เหมาะกับงาน

สิ่งแรกที่ต้องทำคือเลือก "ฮาร์ดแวร์" ให้สอดคล้องกับบทบาทของ Go ในโปรเจกต์คุณ

- **Raspberry Pi (Linux SBC):** เหมาะสำหรับทำ Edge Gateway ที่ต้องประมวลผลข้อมูลจากเซ็นเซอร์หลายตัว หรือเป็น Backend ขนาดเล็ก สามารถรัน Go ได้โดยตรง[reference:0]
- **ESP32 / Arduino (Microcontroller):** หากต้องการควบคุมเซ็นเซอร์หรือแอคทูเอเตอร์โดยตรง แนะนำให้ใช้ **TinyGo** ซึ่งเป็น Go เวอร์ชันสำหรับไมโครคอนโทรลเลอร์[reference:1][reference:2]
- **Computer ปกติ (Windows/macOS/Linux):** เหมาะสำหรับจำลอง (Simulation) หรือพัฒนา Cloud Backend ก่อนนำไป Deploy จริง[reference:3]

2. ติดตั้งเครื่องมือที่จำเป็น

พอเลือกฮาร์ดแวร์ได้แล้ว ขั้นตอนต่อไปคือการติดตั้งเครื่องมือ (Toolchain) ให้พร้อม

- **ติดตั้ง Go ปกติ:** ดาวน์โหลดและติดตั้งจากเว็บไซต์ทางการของ Go[reference:4]
- **ติดตั้ง TinyGo (สำหรับ Microcontroller):** ไปที่เว็บไซต์ TinyGo.org ดาวน์โหลดตัวติดตั้งที่ตรงกับระบบปฏิบัติการของคุณ หรือคอมไพล์จาก source code ก็ได้[reference:5][reference:6]
- **ติดตั้ง Cross-compilation Tools (สำหรับ Linux ARM):** การจะข้าม platform (Cross-compile) โปรเจกต์ Go ของคุณบนเครื่อง PC ให้ไปรันบน Raspberry Pi ได้นั้น สิ่งที่ต้องทำมีแค่เปิดใช้งาน Go Modules และใช้ Environment Variables เท่านั้น[reference:7][reference:8]

3. เขียนโปรแกรม Go สำหรับ IoT (Hands-on)

เมื่อเครื่องมือพร้อม ก็ถึงเวลาเขียนโปรแกรมเพื่อสื่อสารกับฮาร์ดแวร์จริง ๆ กันเลย

ตัวเลือกที่ 1: เขียนบน Raspberry Pi โดยตรง (ใช้ Go ปกติ + periph.io)

หากคุณใช้ Raspberry Pi เป็น Edge Gateway ให้เริ่มจากการควบคุมอุปกรณ์พื้นฐานอย่าง LED ก่อน โดยใช้ periph.io library

```
package main

import (
    "log"
    "time"
    "periph.io/x/conn/v3/gpio"
```

```

    "periph.io/x/conn/v3/gpio/gpioreg"
    "periph.io/x/host/v3"
)

func main() {
    // 1. Initialize host
    if _, err := host.Init(); err != nil {
        log.Fatal(err)
    }

    // 2. Get pin by name (BCM 18)
    pin := gpioreg.ByName("GPIO18")
    if pin == nil {
        log.Fatal("Pin not found")
    }

    // 3. Configure as output
    if err := pin.Out(gpio.High); err != nil {
        log.Fatal(err)
    }

    // 4. Blink LED
    for i := 0; i < 5; i++ {
        pin.Set(gpio.High)
        time.Sleep(500 * time.Millisecond)
        pin.Set(gpio.Low)
        time.Sleep(500 * time.Millisecond)
    }
}

```

[14+L11-L17]

วิธีรัน: `go run main.go` (อาจต้องใช้ `sudo` หากเจอ permission error)[reference:9]

ตัวเลือกที่ 2: เขียนบน ESP32 โดยตรง (ใช้ TinyGo + machine package)

หากคุณใช้ ESP32 เป็นอุปกรณ์ปลายทาง ให้เริ่มจากโปรแกรมกะพริบ LED บนบอร์ด (Blink) ก่อน

```

package main

import (
    "machine"
    "time"
)

func main() {
    led := machine.LED // กำหนด pin ของ LED
    led.Configure(machine.PinConfig{Mode: machine.PinOutput})

    for {

```

```

    led.Toggle()
    time.Sleep(time.Second)
}
}

```

[11+L17-L19]

วิธีรัน: `tinygo flash -target=esp32c3-supermini blink.go`[reference:10]

4. เชื่อมต่ออุปกรณ์กับระบบ IoT (MQTT)

เมื่อควบคุมฮาร์ดแวร์ได้แล้ว ขั้นตอนต่อไปคือการส่งข้อมูลขึ้น Cloud หรือระบบอื่น ๆ โดยใช้ MQTT

- **ติดตั้ง MQTT Client:** ในโปรเจกต์ Go ของคุณ ให้ใช้คำสั่ง `go get github.com/eclipse/paho.mqtt.golang`[reference:11]
- **เขียน Publisher (อุปกรณ์):** ตัวอุปกรณ์จะทำหน้าที่อ่านค่าจากเซ็นเซอร์แล้วส่งข้อมูล JSON ออกไปยัง Topic `sensor/temperature`[reference:12]

5. เริ่มต้นสร้าง Full-stack IoT Project

เมื่อเข้าใจพื้นฐานครบถ้วนแล้ว ลองนำความรู้ทั้งหมดมาประกอบเป็นโปรเจกต์เต็มรูปแบบดู

- **โครงสร้าง 3 ชั้น:** สร้างระบบ IoT แบบมีอาชีวะด้วยสถาปัตยกรรม三层架构[reference:13]
 - **** (Perception Layer)**:** เขียนโปรแกรมด้วย **TinyGo** รันบน ESP32 เพื่ออ่านค่าจากเซ็นเซอร์ DHT11[reference:14]
 - **** (Network Layer)**:** ตัว ESP32 จะส่งข้อมูลขึ้น Cloud ผ่าน MQTT protocol[reference:15]
 - **** (Application Layer)**:** สร้าง Web Dashboard ด้วย Go (ใช้ Gin/Echo) เพื่อแสดงผลข้อมูลแบบ Real-time[reference:16]
- **Gobot Framework:** สำหรับโปรเจกต์ที่ซับซ้อนขึ้น เช่น การควบคุมหุ่นยนต์หรือ Drone แนะนำให้ใช้ **Gobot framework** ซึ่งเป็นเครื่องมือระดับสูงสำหรับ Go โดยเฉพาะ[reference:17]

 แหล่งเรียนรู้เพิ่มเติม

- **Official Documentation:** golang.org/doc, tinygo.org
- **GitHub Repositories:** [TinyGo](#), [Gobot](#), [Periph.io](#)
- **Tutorials:** [datasea.cn Go IoT tutorials](#)[reference:18], [MQTT with Go](#)[reference:19]

แนะนำให้เริ่มจากโปรเจกต์ **"Environmental Monitoring Node"** ที่ใช้ ESP32 อ่านค่าจากเซ็นเซอร์ DHT11 ส่งข้อมูลผ่าน MQTT ขึ้นไปยัง Broker สาธารณะ (เช่น EMQX) จากนั้นใช้ Go เขียน Service เพื่อ Subscribe ข้อมูลและแสดงผลผ่าน Web Dashboard วิธีนี้จะช่วยให้คุณเข้าใจภาพรวมของระบบ IoT ได้อย่างเป็นระบบ

Remark

 Nil ในภาษา Go (Golang)

 Nil คืออะไร?

Nil คือค่าศูนย์ (zero value) ของประเภทข้อมูลที่มีความสามารถในการอ้างอิงถึงพื้นที่หน่วยความจำอื่นๆ (reference types) ในภาษา Go ซึ่งได้แก่:

- **Pointer** (***T**) → ตัวชี้ไปยังค่าของชนิด **T**
- **Slice** (**[]T**) → ส่วนตัดตอนของอาร์เรย์ (ขนาดปรับเปลี่ยนได้)
- **Map** (**map[K]V**) → แผนที่จับคู่คีย์ **K** กับค่า **V**
- **Channel** (**chan T**) → ช่องทางรับส่งข้อมูลชนิด **T** ระหว่าง goroutine
- **Function** (**func**) → ฟังก์ชัน
- **Interface** (**interface{}**) → ชุดของเมธอด (ข้อกำหนด) ที่ชนิดอื่นต้องทำให้ได้

nil หมายถึง "ไม่มีค่า" หรือ "ไม่ได้ชี้ไปที่สิ่งใด" เหมือน **null** ในภาษาอื่นๆ แต่มีความปลอดภัยและพฤติกรรมที่แตกต่างกันในแต่ละประเภท

Nil มีกี่แบบ?

ใน Go **nil** ไม่ใช่ประเภทในตัวของมันเอง แต่เป็นค่าที่กำหนดไว้ล่วงหน้า (predeclared identifier) ซึ่งสามารถกำหนดให้กับตัวแปรของ **6 ประเภท** ดังนี้:

ลำดับ	ประเภทข้อมูล	ตัวอย่างการประกาศ nil	หมายเหตุ
1	Pointer	<code>var p *int = nil</code>	ชี้ไปที่ nowhere
2	Slice	<code>var s []int = nil</code>	ไม่มี underlying array
3	Map	<code>var m map[string]int = nil</code>	ไม่มี hash table
4	Channel	<code>var ch chan int = nil</code>	ไม่มีการเชื่อมต่อ
5	Function	<code>var f func() = nil</code>	ไม่มีโค้ดให้เรียก
6	Interface	<code>var i interface{} = nil</code>	ทั้ง dynamic type และ dynamic value เป็น nil

ข้อสำคัญ: แม้ nil จะเป็นค่าเดียวกัน แต่การใช้งานกับแต่ละประเภทให้ผลลัพธ์ต่างกัน เช่น การอ่านจาก nil map ได้ค่า zero value แต่การเขียนลง nil map จะ panic

Panic ในภาษา Go คือ **ภาวะที่โปรแกรมทำงานต่อไปไม่ได้** เนื่องจากเกิดข้อผิดพลาดร้ายแรง (runtime error) หรือถูกเรียกโดยตรง (**panic()**) ส่งผลให้:

- ฟังก์ชันปัจจุบันหยุดทำงานทันที
- แทนเรียก (stack) ถูกคลี่กลับ (unwind) พร้อมรัน **defer** ตามลำดับ
- หากไม่มี **recover()** มาจับไว้ โปรแกรมจะ **จบการทำงานทันที** พร้อมแสดง stack trace

เปรียบเทียบง่ายๆ:

- **error** = ปัญหาที่คาดเดาได้ (เช่น หาไฟล์ไม่เจอ) → จัดการได้
- **panic** = ปัญหาที่ไม่ควรเกิดขึ้น (เช่น อ่านค่า nil pointer) → โปรแกรมพัง

Nil ใช้อย่างไร? ในกรณีไหน?

1. Nil Pointer

- **ใช้เมื่อ:** ต้องการแสดงว่า pointer ยังไม่ได้ชี้ไปที่ struct หรือตัวแปรใดๆ
- **การใช้งาน:** ต้องตรวจสอบ nil ก่อน dereference (*p) มิฉะนั้นจะ panic

```
var p *int
fmt.Println(p == nil) // true
// *p = 10 // panic: runtime error: invalid memory address
```

2. Nil Slice

- **ใช้เป็น:** ค่าเริ่มต้นของ slice ที่ยังไม่มี element (เทียบเท่ากับ empty slice แต่ต่างกันว่า nil slice มี length=0, capacity=0)
- **การใช้งาน:** สามารถใช้ **append** กับ nil slice ได้ (Go จะสร้าง underlying array ให้อัตโนมัติ)

```
var s []int // nil slice
s = append(s, 1) // ทำงานได้ดี
fmt.Println(s) // [1]
```

3. Nil Map

- **ใช้เมื่อ:** ประกาศ map โดยไม่ใช่ **make** หรือ literal
- **ข้อควรระวัง:** อ่านได้ (คืนค่า zero value) แต่ **เขียนไม่ได้** (จะ panic)

```
var m map[string]int
fmt.Println(m["key"]) // 0 (ไม่ panic)
// m["key"] = 1 // panic: assignment to entry in nil map
```

4. Nil Channel

- **ใช้เมื่อ:** ต้องการปิดการสื่อสาร หรือยังไม่ได้ initialize channel
- **พฤติกรรม:** การส่ง (**ch <- v**) และรับ (**<-ch**) จะ **block ตลอดไป** (deadlock ถ้าไม่มี goroutine อื่น)

5. Nil Function

- **ใช้เมื่อ:** ตัวแปรฟังก์ชันยังไม่ได้ถูกกำหนด (ใช้กับ strategy pattern หรือ callback)
- **การใช้งาน:** ต้องตรวจสอบ nil ก่อนเรียก มิฉะนั้นจะ panic

6. Nil Interface

- **กรณีซับซ้อน:** interface ที่มี dynamic type = nil แต่ dynamic value ≠ nil จะไม่เท่ากับ nil โดยตรง
- **การใช้:** **if err == nil** หรือ **if v == nil** เฉพาะเมื่อทราบว่า interface ไม่ได้เก็บ concrete type ไว้

แผนภาพ Dataflow (Flowchart TB) – การทำงานกับ Nil ใน Go

flowchart TB

```
start[เริ่มต้น] --> declare{ประกาศตัวแปร<br>ประเภท reference types}

declare --> |Pointer| ptrNil[var p *int = nil]
declare --> |Slice| sliceNil[var s []int = nil]
declare --> |Map| mapNil[var m mapK V = nil]
declare --> |Channel| chNil[var ch chan int = nil]
declare --> |Func| funcNil[var f func = nil]
declare --> |Interface| ifaceNil[var i interface = nil]

ptrNil --> ptrUse{ใช้งาน pointer}
ptrUse --> |dereference *p| panic1[panic: nil pointer dereference]
ptrUse --> |เปรียบเทียบ nil| okPtr[if p != nil →ปลอดภัย]

sliceNil --> sliceUse{ใช้งาน slice}
sliceUse --> |len(s), cap(s)| zeroLen[ได้ 0, 0]
sliceUse --> |append(s, x)| newSlice[สร้าง slice ใหม่ ทำงานได้]
sliceUse --> |s[index]| panic2[panic: index out of range]

mapNil --> mapUse{ใช้งาน map}
mapUse --> |อ่าน m[key]| zeroVal[ได้ zero value ของ value type]
mapUse --> |เขียน m[key]=v| panic3[panic: assignment to nil map]
mapUse --> |len(m)| zeroLenMap[ได้ 0]

chNil --> chUse{ใช้งาน channel}
chUse --> |ส่ง ch <- v| block1[block ตลอดไป]
chUse --> |รับ <-ch| block2[block ตลอดไป]
chUse --> |close(ch)| panic4[panic: close of nil channel]

funcNil --> funcUse{เรียกใช้ฟังก์ชัน}
funcUse --> |f()| panic5[panic: nil function call]
funcUse --> |ตรวจสอบ nil| safeCall[if f != nil →เรียกได้]

ifaceNil --> ifaceUse{ใช้งาน interface}
ifaceUse --> |เปรียบเทียบ i == nil| trueNil[true]
ifaceUse --> |type assertion| panic6[panic ถ้า type ไม่ตรง]
ifaceUse --> |เก็บค่า non-nil| special[interface ไม่เป็น nil แม้ value เป็น nil]

okPtr --> end1[จบ]
newSlice --> end2[จบ]
zeroLen --> end3[จบ]
zeroVal --> end4[จบ]
block1 --> deadlock[deadlock]
safeCall --> end5[จบ]
trueNil --> end6[จบ]

style panic1 fill:#ffcccc,stroke:#ff0000
style panic2 fill:#ffcccc,stroke:#ff0000
style panic3 fill:#ffcccc,stroke:#ff0000
style panic4 fill:#ffcccc,stroke:#ff0000
style panic5 fill:#ffcccc,stroke:#ff0000
style panic6 fill:#ffcccc,stroke:#ff0000
```

```
style deadlock fill:#ffcccc,stroke:#ff0000
style special fill:#ffffcc,stroke:#ffaa00
```

 ตัวอย่างโค้ดจริง พร้อมคำอธิบายแต่ละจุด